1- Fundamentos da Agilidade



A imagem compara de forma simples duas formas de desenvolver software:

- **Lado Esquerdo (Waterfall / Cascata):** Mostra um processo rígido em formato de escada, onde as etapas seguem uma ordem fixa (de cima para baixo) e o produto só é entregue pronto no final.

- **Lado Direito (Ágil):** Mostra um ciclo contínuo e flexível (focado em Sprints), onde a equipe trabalha junta, recebe feedback constante e entrega versões funcionais do software aos poucos.

1. Introdução: O Contexto e a Crise do Software

- **O Modelo Tradicional de Engenharia:** Entre as décadas de 1960 e 1980, o desenvolvimento de software tentou espelhar os processos industriais e da engenharia civil. O padrão dominante era o **Modelo em Cascata (*Waterfall*)**, caracterizado por fases lineares, sequenciais e rígidas: levantamento completo de requisitos, projeto de arquitetura, codificação, testes e implantação final. Cada

etapa precisava ser 100% concluída, aprovada e documentada antes de a seguinte ter início.

- **A Ilusão da Previsibilidade:** A premissa central e fatal do modelo em cascata era acreditar que os seres humanos eram capazes de antecipar e congelar todas as necessidades, problemas e tecnologias de um projeto no primeiro dia de planejamento. Acreditava-se que o software era como construir uma ponte ou fabricar um carro, onde o plano inicial nunca muda.
- **A Origem da Crise do Software:** O termo ganhou forte repercussão internacional na Conferência da OTAN de 1968, em Garmisch, para nomear uma falha sistêmica e generalizada na indústria de TI. Os projetos da época eram sinônimos de fracasso crônico: estouros absurdos de prazos (atrasos de anos), orçamentos milionários estourados, sistemas repletos de bugs críticos que quebravam em produção, ou softwares inteiros que eram cancelados antes mesmo do lançamento.
- **O Fator Incerteza e Mudança:** O mercado, a economia, a tecnologia e o comportamento dos usuários mudam de forma veloz e imprevisível. No modelo tradicional, como o ciclo de desenvolvimento durava anos, o mundo mudava drasticamente enquanto o software era construído. O resultado? O sistema ficava pronto exatamente como planejado anos antes, mas totalmente obsoleto e desconectado da realidade do negócio. O planejamento rígido transformou-se no maior inimigo da relevância do produto.

2. Desenvolvimento Tradicional vs. Desenvolvimento Ágil

Para dominar a matéria em provas e entender a virada de chave do mercado, é essencial compreender o contraste estrutural, prático e filosófico entre as duas abordagens:

- **Modelo Tradicional (Cascata / *Waterfall*):**
    - **Dinâmica de Execução:** Sequencial, descendente e inflexível. O trabalho flui em uma única direção. Modificações em fases iniciais (como alterar um requisito) após o projeto já estar avançado geram custos altíssimos, desperdício de meses de trabalho e retrabalho monumental.
    - **Foco Central:** Adesão estrita ao plano original, contratos engessados de escopo fechado e documentação exaustiva prévia. O sucesso era medido por "seguir o plano à risca", mesmo que o plano estivesse errado.
    - **Papel do Cliente:** Participação intensa e pesada apenas no início (para a assinatura do escopo e requisitos) e no encerramento (no aceite e recebimento do sistema pronto). O cliente fica completamente isolado do processo produtivo intermediário, gerando a clássica surpresa desagradável no final.

- **Modelo Ágil:**
    - **Dinâmica de Execução:** Iterativa e incremental. O trabalho total é fracionado em ciclos curtos, fixos e recorrentes (geralmente chamados de *sprints*, que duram de 1 a 4 semanas). Em cada ciclo, a equipe entrega um incremento de software real, testado, integrado e totalmente funcional.
    - **Foco Central:** Adaptação ágil a mudanças de cenário, interações humanas, colaboração direta e entrega constante de valor real. O sucesso é medido pela capacidade de entregar valor real ao cliente e responder rapidamente a novas necessidades.
    - **Papel do Cliente:** Presente, ativo, colaborativo e consultado de forma contínua a cada final de ciclo (*Sprint Review*) para validar as entregas, testar o produto e redirecionar os próximos passos com base na realidade.

3. Os Benefícios da Agilidade

A transição para metodologias ágeis transforma radicalmente a entrega de valor por meio de vantagens estratégicas profundas e mensuráveis:

- **Mitigação Precoce de Riscos:** Como as entregas, os testes e as integrações ocorrem continuamente a cada ciclo curto, falhas arquiteturais, problemas de integração e bugs graves são descobertos e corrigidos nas primeiras semanas de projeto. Isso impede que esses erros se acumulem e se transformem em uma catástrofe irreversível na entrega final.
- **Retorno sobre o Investimento (ROI) Antecipado:** No modelo ágil, o cliente não precisa aguardar o encerramento de todo o cronograma do projeto (que poderia levar de 1 a 2 anos) para usufruir de resultados. Cada ciclo curto disponibiliza funcionalidades úteis e operacionais, gerando valor imediato ao negócio e permitindo que o produto comece a se pagar antes mesmo do fim do desenvolvimento total.
- **Alinhamento Contínuo com o Usuário e Redução de Desperdício:** Com validações frequentes junto aos usuários finais, o produto evolui moldado às reais necessidades do mercado. Isso mitiga o risco clássico de desenvolver um sistema perfeito em teoria, mas que se torna inútil na prática por não atender ao que o mercado realmente queria.
- **Flexibilidade e Competitividade de Mercado:** Equipes ágeis possuem altíssima capacidade de pivotar e incorporar novas demandas tecnológicas, exigências legais ou respostas estratégicas à concorrência. Elas fazem isso de forma orgânica, sem a burocracia imensa e engessada de repactuação de contratos globais do modelo tradicional.
- **Sustentabilidade e Engajamento da Equipe:** A autonomia, a transparência, a colaboração horizontal e a auto-organização promovidas pela agilidade elevam

drasticamente o moral, a motivação e a sinergia dos profissionais envolvidos, combatendo o *burnout* e reduzindo a rotatividade (*turnover*) nas equipes técnicas.

FONTES UTILIZADAS:

- **Relatório da Conferência da OTAN (1968):** NATO Software Engineering Conference 1968 Report (PDF)
- **Artigo Original do Modelo Cascata (Winston W. Royce, 1970):** Managing the Development of Large Software Systems (PDF)
- **Manifesto para o Desenvolvimento Ágil de Software (2001):** Manifesto Ágil (Português)
- **Guia Oficial do Scrum:** The Scrum Guide (Português)
- **Engenharia de Software Moderna (Prof. Marco Tulio Valente - UFMG):** Capítulo 1: Introdução

2- Manifesto Ágil



A imagem demonstra a comparação entre os 4 valores fundamentais do manifesto ágil

1. Indivíduos e Interações: A comunicação direta entre as pessoas e a auto-organização da equipa superam o uso de ferramentas ou processos engessados.

2. Software em Funcionamento: Entregar código testado, integrado e funcional gera valor real, enquanto documentações longas ficam obsoletas.

3. Colaboração com o Cliente: Parceria contínua para ajustar o *backlog* e o produto substitui contratos rígidos de escopo fechado.

4. Responder a Mudanças: *Feedback loops* curtos (Sprints) permitem adaptar o projeto rapidamente, em vez de seguir cegamente um plano preditivo.

1. Contexto Histórico e a Criação do Manifesto

- **O Encontro de Snowbird (2001):** Em fevereiro de 2001, um grupo de 17 desenvolvedores, consultores e pioneiros da indústria de software (conhecidos posteriormente como os "Organizadores da Agilidade") reuniu-se em uma estação de esqui em Snowbird, Utah, nos Estados Unidos. O objetivo comum era debater alternativas aos processos de desenvolvimento de software tradicionais, que eram pesados, burocráticos, engessados e focados excessivamente em documentações exaustivas e planejamento rígido.
- **A Nascente da Filosofia:** Descontentes com a burocracia do modelo em cascata (*Waterfall*), esse grupo percebeu que todos compartilhavam de princípios

semelhantes focados em colaboração, flexibilidade e entrega rápida de valor. Dessa cúpula histórica nasceu o **Manifesto Ágil** (*Agile Manifesto*), acompanhado de sua declaração oficial e de um conjunto de diretrizes práticas.

- **A Frase Emblemática:** A essência do documento pode ser resumida na frase que define a sua postura diante da tecnologia e dos negócios: *"Estamos descobrindo maneiras melhores de desenvolver software, fazendo-o nós mesmos e ajudando outros a fazerem o mesmo"*. O manifesto não inventou ferramentas novas do zero, mas uniu e nomeou práticas que já funcionavam bem em equipes de alta performance.

2. Os Quatro Valores do Manifesto Ágil

O coração do Manifesto Ágil é composto por quatro valores fundamentais. É crucial entender que **a agilidade não despreza os itens da direita**, mas reconhece que **os itens da esquerda possuem muito mais valor** para o sucesso de um projeto de software:

- **1. Indivíduos e interações mais que processos e ferramentas:**
  - *Explicação:* Por mais avançadas que sejam as ferramentas de gestão ou rigorosos que sejam os processos, o sucesso de um projeto depende primordialmente das pessoas envolvidas e de como elas se comunicam. Processos rígidos não resolvem falhas de comunicação humana.
- **2. Software em funcionamento mais que documentação abrangente:**
  - *Explicação:* A documentação tem o seu valor para registro histórico e técnico, mas o que realmente gera valor para o cliente e valida o negócio é um sistema funcional rodando nas mãos do usuário. O progresso não deve ser medido por pilhas de papéis escritos, mas por partes do software que já funcionam na prática.
- **3. Colaboração com o cliente mais que negociação de contratos:**
  - *Explicação:* Em vez de tratar o cliente como um oponente comercial em uma relação de "ganha-perde" baseada em contratos engessados e punitivos, a agilidade propõe uma parceria contínua. O cliente faz parte do time do projeto, ajustando rotas e prioridades lado a lado com a equipe de desenvolvimento.
- **4. Responder a mudanças mais que seguir um plano:**
  - *Explicação:* Ter um plano inicial estruturado é importante, mas ser escravo dele quando a realidade do mercado muda é um erro fatal. O modelo ágil valoriza a capacidade de pivotar, adaptar-se e abraçar novas oportunidades ou correções de rumo a qualquer momento, encarando a mudança como uma vantagem competitiva e não como um problema.

3. Os Doze Princípios do Manifesto Ágil

Para guiar a aplicação prática dos quatro valores, o Manifesto Ágil estabeleceu doze princípios fundamentais que norteiam o comportamento das equipes ágeis:

- **1. Nossa maior prioridade é satisfazer o cliente através da entrega contínua e adiantada de software com valor agregado.**
    - *Explicação:* O cliente não precisa esperar o fim do projeto para ver resultados; entregas frequentes e úteis geram satisfação e retorno financeiro rápido.
- **2. Aceitar mudanças de requisitos, mesmo tarde no desenvolvimento.** Processos ágeis tiram vantagem das mudanças visando a vantagem competitiva para o cliente.
    - *Explicação:* Mudanças de rumo não são vistas como falhas de planejamento, mas como oportunidades de entregar um produto final muito melhor e mais alinhado com o mercado atual.
- **3. Entregar software funcionando com frequência, na escala de poucas semanas até poucos meses, com preferência para os curtos períodos.**
    - *Explicação:* Ciclos curtos reduzem drasticamente o escopo de erros, facilitam testes e garantem um fluxo constante de valor real sendo entregue.
- **4. Pessoas de negócios e desenvolvedores devem trabalhar em conjunto diariamente, durante todo o projeto.**
    - *Explicação:* O fim dos "silos" organizacionais. Quem entende do negócio e quem codifica a tecnologia precisam conversar todos os dias para evitar mal-entendidos e retrabalho.
- **5. Construir projetos ao redor de indivíduos motivados.** Dê a eles o ambiente e o suporte necessário e confie neles para fazer o trabalho.
    - *Explicação:* A agilidade confia na autonomia e na competência técnica da equipe, fornecendo suporte institucional em vez de microgerenciamento e controle excessivo.
- **6. O método mais eficiente e eficaz de transmitir informações para e entre uma equipe de desenvolvimento é a conversa cara a cara.**
    - *Explicação:* Embora e-mails, relatórios e chats sejam úteis, a comunicação verbal direta (presencial ou por vídeo) é infinitamente mais rápida, rica em nuances e evita ruídos de interpretação.
- **7. Software funcionando é a medida primária de progresso.**
    - *Explicação:* O termômetro de avanço de um projeto ágil não é a quantidade de relatórios gerados ou o cumprimento de planilhas de tarefas, mas sim a quantidade de funcionalidades reais operando.

- **8. Os processos ágeis promovem desenvolvimento sustentável.** Os patrocinadores, desenvolvedores e usuários devem ser capazes de manter um ritmo constante indefinidamente.
    - *Explicação:* A agilidade combate a cultura de horas extras abusivas (*crunch time*), promovendo cadências saudáveis para evitar o desgaste físico e mental (*burnout*) da equipe.
- **9. Contínua atenção à excelência técnica e ao bom design aumenta a agilidade.**
    - *Explicação:* Para continuar ágil no futuro, o código precisa ser limpo, bem estruturado e testável. Códigos mal feitos geram "dívida técnica" que engessa o projeto.
- **10. Simplicidade — a arte de maximizar a quantidade de trabalho que não precisou ser feito — é essencial.**
    - *Explicação:* Fazer o que é estritamente necessário para entregar valor, evitando desperdícios com funcionalidades complexas que ninguém pediu ou usará.
- **11. As melhores arquiteturas, requisitos e designs emanam de equipes auto-organizáveis.**
    - *Explicação:* As equipes que colocam a mão na massa são as mais capacitadas para decidir *como* o trabalho deve ser feito, tendo autonomia para organizar suas próprias tarefas.
- **12. Em intervalos regulares, a equipe reflete sobre como se tornar mais eficaz, então reflete e ajusta seu comportamento de acordo.**
    - *Explicação:* O processo de melhoria contínua (através de reuniões de retrospectiva). A equipe para periodicamente para analisar o que deu errado, o que deu certo e como pode evoluir no próximo ciclo.

FONTES UTILIZADAS

- **Manifesto para o Desenvolvimento Ágil de Software (Site Oficial):** Manifesto Ágil - Declaração Oficial (Português)

- **Os 12 Princípios do Manifesto Ágil (Site Oficial):** Os Doze Princípios do Software Ágil (Português)

- **História do Encontro de Snowbird (Site Oficial):** History: The Agile Manifesto

- **Engenharia de Software Moderna (Prof. Marco Tulio Valente - UFMG):** Capítulo 2: Processos Ágeis

3- Introdução ao Scrum



Este diagrama apresenta a visão geral do **Ciclo Scrum** e como as etapas do processo se conectam com seus fundamentos empíricos:

- **Entradas e Planejamento:** Organização e ordenação de prioridades no *Product Backlog*, seguido da escolha dos itens no *Sprint Planning* para definir o *Sprint Backlog*.

- **Execução e Acompanhamento:** Desenvolvimento ao longo da *Sprint* com reuniões diárias (*Daily Scrum*) de 15 minutos para alinhar o progresso e remover impedimentos.

- **Entrega e Aprendizado:** Validação do *Incremento* funcional na *Sprint Review* com os stakeholders e identificação de melhorias do processo na *Sprint Retrospective*.

Exemplos dos 3 Pilares:

- **Transparência:** Manter o Quadro Kanban visível para toda a equipe e stakeholders.
- **Inspeção:** Avaliar o progresso diariamente no *Daily Scrum* e na *Sprint Review*.
- **Adaptação:** Corrigir falhas e ajustar a rotina de trabalho durante a *Sprint Retrospective*.

Capítulo 3 – O que é Scrum

Agora que você já sabe qual problema o Scrum tenta resolver e de que movimento ele faz parte, podemos olhar para ele diretamente. Neste capítulo, vamos entender a definição oficial, conhecer a história do Scrum e ver por que ele se tornou tão importante no desenvolvimento de software.

3.1 A definição oficial, palavra por palavra

O Scrum Guide define o Scrum como um **framework leve que ajuda pessoas, times e organizações a gerar valor por meio de soluções adaptativas para problemas complexos**. Essa frase é curta, mas cada pedaço dela carrega uma ideia importante, então vale a pena desmontá-la.

Framework: como vimos no capítulo anterior, é uma estrutura que define os elementos essenciais e as regras entre eles, sem detalhar todas as técnicas. O Scrum diz que haverá uma Sprint, um Product Owner, um Product Backlog e assim por diante, mas não diz como você deve programar, testar ou desenhar telas.

Leve: o Scrum tem poucos elementos. O Scrum Guide inteiro cabe em menos de quinze páginas. Isso é proposital. Quanto menos regras um framework tem, menos ele atrapalha a equipe e mais espaço sobra para que ela descubra o que funciona no seu contexto.

Pessoas, times e organizações: o Scrum não é só para programadores. Ele envolve a equipe que faz o trabalho, mas também as pessoas interessadas no produto e a organização como um todo. Uma empresa que adota Scrum só no time de desenvolvimento, mas mantém o resto da organização funcionando de forma rígida, costuma ter dificuldades.

Gerar valor: o objetivo não é “cumprir tarefas” nem “seguir o processo”, e sim produzir algo que tenha valor para alguém. Valor pode ser um usuário conseguindo pedir comida mais rápido, um restaurante vendendo mais ou uma empresa reduzindo custos. Uma equipe que entrega muitas funcionalidades inúteis está ocupada, mas não está gerando valor.

Soluções adaptativas para problemas complexos: retoma tudo o que vimos no Capítulo 1. Como o problema é complexo, a solução não pode ser fixada no início; ela precisa se adaptar à medida que aprendemos.

Logo após a definição, o Scrum Guide resume o funcionamento do Scrum em quatro passos, num ambiente que o Scrum Master ajuda a criar. Em linguagem simples, é assim: primeiro, o **Product Owner** organiza o trabalho necessário para resolver um problema complexo em uma lista chamada **Product Backlog**. Segundo, o **Scrum Team** transforma uma parte desse trabalho em um **Increment** de valor durante uma **Sprint**. Terceiro, o Scrum Team e as partes interessadas inspecionam os resultados e fazem ajustes para a próxima Sprint. Quarto, o ciclo se repete.

Não se preocupe se esses termos ainda parecem estranhos. Cada um deles terá uma seção própria mais adiante. Por enquanto, guarde a ideia central: existe uma lista ordenada do que precisa ser feito, a

equipe pega uma parte dessa lista, transforma em algo utilizável dentro de um ciclo curto, todos olham o resultado, ajustam o rumo e começam outro ciclo.

O Scrum Guide acrescenta mais duas observações que você deve guardar. A primeira é que o Scrum é **propositalmente incompleto**: ele define apenas as partes necessárias para aplicar a teoria do Scrum, e a inteligência coletiva das pessoas que o usam completa o restante. A segunda é que os elementos do Scrum (papéis, eventos, artefatos e regras) são **imutáveis**. Isso não significa que a equipe não possa adicionar práticas; significa que, se você retirar ou alterar partes essenciais, o resultado pode até funcionar, mas não é mais Scrum. O guia diz que o Scrum só existe em sua totalidade. Por exemplo, uma equipe que trabalha em ciclos de duas semanas, mas nunca faz a Retrospective e não tem um Product Owner, está usando algumas ideias do Scrum, mas não está usando Scrum.

3.2 De onde vem o nome e como o Scrum surgiu

O nome Scrum vem do **rugby**. No rugby, o *scrum* é uma formação em que os jogadores dos dois times se agrupam, abraçados e empurrando juntos, para disputar a posse da bola. A imagem transmite a ideia de um time inteiro se movendo como uma unidade, em vez de cada jogador fazer sua parte isoladamente.

A origem dessa metáfora está num artigo publicado em 1986 na revista Harvard Business Review pelos pesquisadores japoneses **Hirotaka Takeuchi e Ikujiro Nonaka**, intitulado *The New New Product Development Game*. Eles estudaram empresas que desenvolviam produtos inovadores com rapidez, como fabricantes de câmeras, copiadoras e automóveis, e perceberam que elas não trabalhavam como numa corrida de revezamento, em que um corredor entrega o bastão ao próximo (exatamente como as fases da cascata). Em vez disso, essas empresas usavam equipes pequenas e multidisciplinares, que trabalhavam juntas do início ao fim, sobrepondo as fases e passando a bola de um lado para o outro, como no rugby. Os autores usaram a palavra *scrum* para descrever esse estilo.

No início dos anos 1990, **Jeff Sutherland**, que trabalhava com desenvolvimento de software nos Estados Unidos, se inspirou nesse artigo e em outras ideias para organizar uma equipe de forma diferente, em ciclos curtos e com o time trabalhando junto. Na mesma época, **Ken Schwaber** vinha estudando por que os processos tradicionais de software falhavam e chegou a conclusões parecidas, apoiado na ideia de controle empírico de processos, que veremos no próximo capítulo. Os dois passaram a colaborar e, em **1995**, apresentaram juntos o Scrum pela primeira vez numa conferência de software chamada OOPSLA. Em 2001, ambos estavam entre os signatários do Manifesto Ágil, e o Scrum passou a ser visto como uma das principais formas de colocar os valores ágeis em prática.

3.3 O Scrum Guide e sua evolução

Durante muitos anos, o Scrum foi transmitido por livros, cursos e artigos, e cada autor descrevia o framework de um jeito um pouco diferente. Para criar uma referência única, Schwaber e Sutherland escreveram em **2010** a primeira versão do **Scrum Guide**, que eles chamam de “o guia definitivo do Scrum” ou “as regras do jogo”. O guia é gratuito e está disponível em diversos idiomas, inclusive em português, no site scrumguides.org.

O guia foi atualizado algumas vezes desde então: em 2011, 2013, 2016 (quando foram incluídos os cinco valores do Scrum), 2017 e **2020**. A versão de 2020 é a atual e oficial, e trouxe mudanças importantes que você vai encontrar ao longo desta apostila. Entre elas, o guia ficou menos prescritivo, ou seja, passou a dar menos receitas prontas e mais espaço para a equipe decidir; deixou de usar linguagem específica de TI, para facilitar o uso do Scrum em outras áreas; trocou a ideia de um "Time de Desenvolvimento" separado dentro do Scrum Team pela ideia de **um único time**, no qual as pessoas que constroem o produto são chamadas de **Developers**; introduziu o **Product Goal** (o objetivo de longo prazo do produto); e associou a cada artefato um **compromisso**, assunto do Capítulo 7.

Essa história ajuda a entender um fenômeno comum: muitos materiais na internet, e até em empresas, ainda descrevem o Scrum como ele era em 2013 ou 2017. Por isso você pode encontrar referências a "Time de Desenvolvimento", às "três perguntas da Daily" ou à ideia de que o Scrum Master "remove impedimentos" pessoalmente. Quando houver divergência, a referência válida é o Scrum Guide de 2020.

3.4 Por que o Scrum é tão usado no desenvolvimento de software

O Scrum Guide deixa claro que o Scrum pode ser usado em vários tipos de trabalho, não só em software. Ainda assim, foi no desenvolvimento de software que ele nasceu e onde ele é mais difundido, e pesquisas de mercado apontam há anos o Scrum como o framework ágil mais adotado. Há boas razões para isso.

A primeira razão é que o Scrum **reduz o risco**. Como a equipe entrega algo utilizável a cada ciclo de no máximo um mês, o pior cenário possível é desperdiçar um ciclo, e não um ano inteiro. Se a equipe do aplicativo de pedidos descobre na terceira Sprint que os usuários quase não usam a função de "favoritar pratos", ela pode parar de investir nisso imediatamente.

A segunda razão é que o Scrum **traz o valor mais cedo**. Como o trabalho é ordenado pelo valor, as funcionalidades mais importantes tendem a ficar prontas primeiro. Às vezes, um produto com 30% das funcionalidades planejadas já resolve 80% do problema do cliente e pode ser lançado.

A terceira razão é a **transparência**. No Scrum, todos podem ver o que está sendo feito, o que já está pronto e quais problemas surgiram. Isso evita a situação clássica em que o projeto parece "90% pronto" por meses a fio.

A quarta razão é a **capacidade de adaptação**. Mercado, tecnologia e necessidades mudam, e o Scrum oferece momentos regulares para incorporar essas mudanças sem bagunçar o trabalho em andamento.

A quinta razão está nas **pessoas**. O Scrum dá à equipe autonomia para decidir como fazer o trabalho, o que costuma aumentar o engajamento e a qualidade.

É justo, porém, terminar este capítulo com um alerta. O Scrum não é uma fórmula mágica. Ele não resolve sozinho problemas de falta de habilidade técnica, de liderança autoritária ou de metas impossíveis. O que ele faz muito bem é **tornar esses problemas visíveis** rapidamente, para que possam ser tratados. Se a equipe nunca consegue terminar o que planeja, o Scrum vai mostrar isso a cada Sprint. Cabe às pessoas usar essa informação para melhorar. E essa ideia de tornar as coisas visíveis para poder melhorá-las é exatamente o fundamento teórico do Scrum, que estudaremos a seguir.

Os fundamentos do Scrum

Muita gente aprende Scrum decorando a lista de papéis, eventos e artefatos. Isso funciona para passar numa prova rápida, mas não ajuda a usar o Scrum de verdade. Os elementos do Scrum só fazem sentido quando entendemos a teoria por trás deles. Este capítulo trata justamente dessa teoria: o empirismo, o pensamento Lean, os três pilares e os cinco valores. Se você entender bem este capítulo, vai perceber que os capítulos seguintes são apenas a aplicação prática dessas ideias.

4.1 Empirismo e pensamento Lean

O Scrum Guide afirma que o Scrum se baseia em duas linhas de pensamento: o **empirismo** e o **pensamento Lean**.

A palavra empirismo vem da filosofia e tem a ver com experiência. O empirismo afirma que **o conhecimento vem da experiência** e que as **decisões devem ser tomadas com base no que é observado**. Parece óbvio, mas compare com a cascata: nela, as decisões mais importantes (o que construir, como, em quanto tempo) são tomadas no início, quando ainda não há experiência nenhuma com aquele produto. No empirismo, a equipe faz uma pequena parte, observa o resultado e só então decide o próximo passo com base em fatos, não em suposições.

Um exemplo do cotidiano ajuda. Imagine que você vai cozinhar um prato que nunca fez. Uma abordagem é seguir a receita à risca e só provar no final. Outra é ir provando enquanto cozinha: está sem sal, adiciona um pouco; ficou grosso demais, acrescenta água. A segunda abordagem é empírica. Você toma decisões a partir do que observa, e o resultado tende a ser melhor, principalmente quando a receita é nova para você.

No desenvolvimento de software, o equivalente a "provar a comida" é colocar uma versão funcionando nas mãos de alguém e observar o que acontece. A equipe do aplicativo de pedidos pode achar que os usuários vão adorar filtrar restaurantes por tempo de entrega. Em vez de apostar meses nisso, ela entrega uma versão simples do filtro e observa os dados de uso. Se quase ninguém usa, a decisão de investir mais nessa funcionalidade passa a ser baseada em evidência.

Para isso funcionar, o Scrum usa uma abordagem **iterativa e incremental**, termos que você já conhece do Capítulo 2. Segundo o Scrum Guide, essa abordagem serve para **otimizar a previsibilidade e controlar o risco**. Pode parecer contraditório que trabalhar sem um plano completo aumente a previsibilidade, mas pense bem: depois de algumas Sprints, a equipe tem dados reais sobre quanto consegue entregar e sobre o que os usuários valorizam. Esses dados permitem previsões muito mais confiáveis do que uma estimativa feita no escuro, antes de começar.

O **pensamento Lean** vem da indústria, especialmente do sistema de produção da Toyota, e tem como ideia central **reduzir desperdícios e focar no essencial**. Em software, desperdício pode ser uma funcionalidade que ninguém usa, um documento que ninguém lê, código que fica meses parado esperando ser testado, ou uma tarefa refeita porque alguém entendeu errado o que era para fazer.

Quando a equipe do aplicativo decide não construir agora um painel de estatísticas para o restaurante, porque ninguém sabe ainda quais números importam, ela está aplicando o pensamento Lean.

Por fim, o Scrum Guide observa que o Scrum envolve grupos de pessoas que, **coletivamente**, têm todas as habilidades e conhecimentos necessários para fazer o trabalho. Isso será importante quando falarmos do Scrum Team no próximo capítulo.

4.2 Os três pilares: transparência, inspeção e adaptação

O empirismo, na prática, se sustenta em três pilares: **transparência, inspeção e adaptação**. Eles funcionam como uma corrente, em que cada elo depende do anterior.

A **transparência** significa que o processo e o trabalho devem estar **visíveis** tanto para quem faz o trabalho quanto para quem o recebe. No Scrum, decisões importantes são tomadas com base no estado dos três artefatos formais (Product Backlog, Sprint Backlog e Increment, que veremos no Capítulo 7). Se esses artefatos não refletem a realidade, as decisões tomadas a partir deles serão ruins. O Scrum Guide diz que artefatos com baixa transparência levam a decisões que diminuem o valor e aumentam o risco.

Veja um exemplo de falta de transparência. Na equipe do aplicativo, um desenvolvedor marca a tela de pagamento como “pronta”, mas ela ainda não foi testada com cartões de débito. Todos acreditam que o pagamento está concluído, o Product Owner promete ao restaurante que o recurso estará disponível na semana seguinte e, no dia do lançamento, os pagamentos com débito falham. O problema não foi só técnico: foi de transparência. Se o estado real do trabalho estivesse visível, a decisão de prometer o lançamento teria sido outra. É por isso que o Scrum dá tanta importância a uma definição compartilhada do que significa “pronto”, a Definition of Done, que estudaremos adiante.

A **inspeção** significa examinar com frequência e atenção os artefatos e o progresso em direção aos objetivos combinados, para detectar problemas ou desvios indesejados. Para que a inspeção aconteça de forma regular, o Scrum oferece uma cadência, isto é, um ritmo previsível, por meio de seus cinco eventos. O Scrum Guide faz uma ligação importante aqui: **a transparência possibilita a inspeção**. Inspecionar algo que não está visível, ou que está representado de forma enganosa, é inútil e gera desperdício.

A **adaptação** significa ajustar o processo ou o produto quando a inspeção revela que algo saiu dos limites aceitáveis. O Scrum Guide diz que o ajuste deve ser feito **o mais rápido possível**, para minimizar novos desvios. E aqui aparece outra ligação: **a inspeção possibilita a adaptação**, e uma inspeção que não leva a nenhuma adaptação não tem sentido. De nada adianta a equipe perceber todas as semanas que os testes atrasam a entrega se nada for mudado.

O Scrum Guide acrescenta que a adaptação fica mais difícil quando as pessoas envolvidas **não têm autonomia** ou **não são autogerenciadas**. Se toda mudança precisar ser aprovada por três níveis de chefia, a equipe pode até perceber o problema, mas não vai conseguir reagir a tempo. Por isso se espera que o Scrum Team se adapte no momento em que aprende algo novo.

Uma forma simples de memorizar a relação entre os pilares é pensar num painel de carro. A transparência é o painel mostrar corretamente a velocidade e o combustível. A inspeção é o motorista

olhar para o painel com frequência. A adaptação é o motorista reduzir a velocidade ou parar para abastecer quando necessário. Sem um painel confiável, olhar para ele não adianta; olhar e não fazer nada também não.

4.3 Os cinco valores do Scrum

Os pilares explicam a mecânica do empirismo, mas há uma questão humana por trás: para que as pessoas sejam transparentes, inspecionem com honestidade e se adaptem, elas precisam de um ambiente de confiança. É aí que entram os **cinco valores do Scrum**, incluídos no Scrum Guide em 2016: **comprometimento, foco, abertura, respeito e coragem**.

O Scrum Guide explica cada valor de forma bem direta. Com **comprometimento**, o Scrum Team se compromete a alcançar seus objetivos e a apoiar uns aos outros. Repare que o compromisso é com os **objetivos** e com o **time**, não com a entrega de uma quantidade fixa de tarefas a qualquer custo. Com **foco**, a atenção principal está no trabalho da Sprint, para fazer o melhor progresso possível rumo aos objetivos. Com **abertura**, o time e as partes interessadas são abertos sobre o trabalho e os desafios. Com **respeito**, os membros do time se respeitam como pessoas capazes e independentes e são respeitados assim por quem trabalha com eles. Com **coragem**, os membros têm coragem de fazer a coisa certa e de enfrentar problemas difíceis.

Vamos ver como esses valores aparecem no dia a dia da equipe do aplicativo. Quando uma desenvolvedora percebe que a integração com o sistema de pagamento vai demorar muito mais do que o previsto, ela precisa de **coragem** e **abertura** para dizer isso imediatamente na reunião diária, em vez de esconder o problema e torcer para resolver sozinha. Os colegas, por sua vez, demonstram **respeito** ao tratar o aviso como uma informação útil, e não como uma falha pessoal, e demonstram **comprometimento** ao se oferecer para ajudar. Quando alguém de outra área pede para a equipe "só dar uma olhadinha" num relatório que não tem nada a ver com o objetivo da Sprint, o valor do **foco** ajuda a equipe a dizer que isso pode ser conversado com o Product Owner e considerado para depois.

O Scrum Guide observa que, quando esses valores são vividos pelo Scrum Team e pelas pessoas com quem ele trabalha, os pilares do empirismo ganham vida, construindo **confiança**. Perceba a conexão: sem abertura e coragem, não há transparência real; sem respeito, a inspeção vira caça às bruxas; sem comprometimento e foco, a adaptação não acontece. Os valores são, portanto, a base humana que permite que a teoria funcione. O guia também diz que as decisões tomadas, os passos dados e a forma de usar o Scrum devem reforçar esses valores, e não enfraquecê-los.

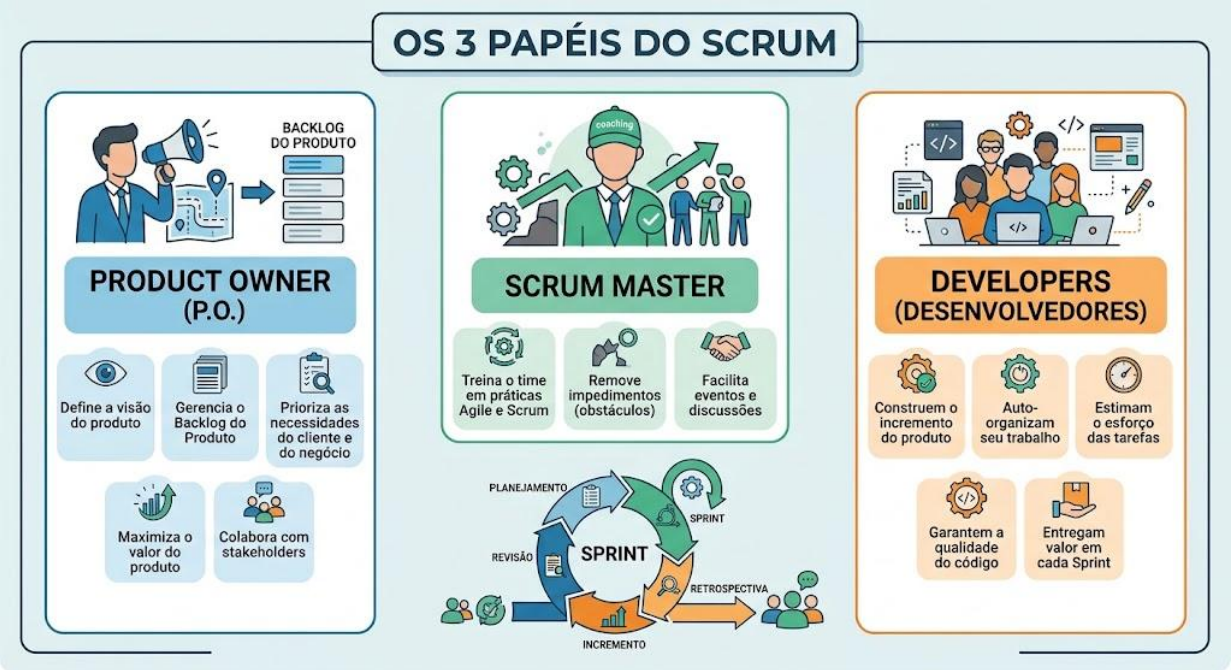
Com os fundamentos estabelecidos, podemos agora conhecer as pessoas que fazem o Scrum acontecer.

Fontes consultadas

- **Scrum Guides** – O Guia Definitivo do Scrum: As Regras do Jogo (novembro de 2020)

- **Scrum Alliance** – Guias e Recursos sobre o Framework Scrum
- **Agile Alliance** – Manifesto para Desenvolvimento Ágil de Software

4- Papéis do Scrum



A imagem apresenta um infográfico educativo focado na estrutura do **Scrum Team**, dividido em três colunas principais:

1. **Product Owner (P.O.):**
    a. **Função:** Define a visão do produto e maximiza o seu valor.
    b. **Atividades:** Gerencia e prioriza o *Product Backlog* e colabora diretamente com os *stakeholders*.

2. **Scrum Master:**
    a. **Função:** Garante a aplicação das práticas ágeis e atua como líder servidor.

b. **Atividades:** Treina a equipa em Scrum, remove impedimentos e facilita os eventos.

3. **Developers (Desenvolvedores):**
   a. **Função:** Criam o incremento funcional do produto com qualidade técnica.
   b. **Atividades:** Auto-organizam o trabalho, estimam tarefas e garantem entregas de valor a cada Sprint.

Papéis do Scrum

1. O Scrum Team

No Scrum, a unidade fundamental é uma equipe pequena chamada **Scrum Team**. Neste capítulo vamos entender como esse time é formado, que características ele precisa ter e quais são as três responsabilidades que existem dentro dele.

O Scrum Team é formado por **Product Owner**, **Scrum Master** e **Developers**, atuando como um grupo **autogerenciado**, **multidisciplinar** e orientado à entrega contínua de valor. Não há hierarquia interna entre os papéis; todos compartilham a responsabilidade pelo **Incremento** e pelo cumprimento da **Sprint Goal**.
O time decide **como** trabalhar, organiza suas próprias atividades e possui todas as competências necessárias para entregar o produto sem depender de equipes externas.

2. Product Owner (PO)

O Product Owner é responsável por **maximizar o valor do produto**. Ele cria e comunica a **visão do produto**, define o **Product Goal** e gerencia o **Product Backlog**, garantindo que os itens estejam claros, priorizados e alinhados às necessidades do negócio, dos usuários e da estratégia organizacional.
Também cabe ao PO aceitar ou rejeitar incrementos, além de manter comunicação constante com stakeholders para alinhar expectativas e decisões.
Entre as habilidades essenciais estão: tomada de decisão, comunicação clara, conhecimento do domínio do produto e capacidade de priorização baseada em valor.

3. Scrum Master (SM)

O Scrum Master é responsável por promover o entendimento e a aplicação correta do Scrum. Atua como facilitador, mentor e agente de mudança, garantindo que os princípios de **transparência, inspeção e adaptação** sejam aplicados.
O SM apoia o Product Owner no gerenciamento do backlog, auxilia os Developers na melhoria de práticas de colaboração e engenharia, remove impedimentos e trabalha com a organização para eliminar barreiras que dificultem a agilidade.

Suas principais habilidades incluem facilitação, coaching, liderança servidora e profundo conhecimento do Scrum Guide.

4. Developers

Os Developers são responsáveis por criar o **Incremento** que atende à **Definition of Done**. Eles planejam o trabalho da Sprint, participam do refinamento do backlog, garantem a qualidade técnica do produto e adaptam o plano conforme surgem novos aprendizados.
Entre suas competências estão colaboração, práticas de engenharia de software, estimativa, planejamento e autogestão.

5. Interações entre os Papéis

A colaboração entre os papéis é essencial para o funcionamento do Scrum:

- **Product Owner e Developers**: trabalham juntos no refinamento do backlog, definição de critérios de aceitação e planejamento da Sprint.
- **Scrum Master e Product Owner**: colaboram para melhorar a transparência do backlog, a comunicação com stakeholders e a aplicação de práticas de priorização.
- **Scrum Master e Developers**: atuam juntos na remoção de impedimentos, facilitação de eventos e melhoria contínua.
- **Todo o Scrum Team** participa dos eventos oficiais: Sprint Planning, Daily Scrum, Sprint Review e Sprint Retrospective.

Uma última observação: o Scrum Guide não proíbe que a mesma pessoa que é Product Owner ou Scrum Master também trabalhe em itens do Sprint Backlog. Quando isso acontece, ela participa da Daily Scrum como Developer. Mas, como cada responsabilidade exige tempo e atenção, acumular funções tem custos, e muitas organizações preferem evitar isso.

Para fixar, a tabela abaixo resume as três responsabilidades.

Responsabilidade	Responde por	Pergunta central
Product Owner	Maximizar o valor do produto e gerenciar o Product Backlog	O que é mais valioso fazer agora?
Developers	Criar o Increment, planejar a Sprint, garantir a qualidade	Como transformamos isso em algo pronto e utilizável?
Scrum Master	Estabelecer o Scrum e a eficácia do Scrum Team	O time está usando bem o Scrum e melhorando a cada ciclo?

FONTES UTILIZADAS

- **The Scrum Guide (Guia Oficial do Scrum):**
- *Descrição:* Documento oficial escrito pelos criadores do framework, Ken Schwaber e Jeff Sutherland, contendo a definição formal do Scrum, o Scrum Team (PO, SM e Developers) e suas responsabilidades.
- **Link oficial:** The Scrum Guide — O Guia do Scrum (Português)
- **Engenharia de Software Moderna (Prof. Marco Tulio Valente - UFMG):**
- *Descrição:* Material acadêmico de referência que detalha a organização de equipes ágeis, os papéis no Scrum e as dinâmicas de desenvolvimento de software.
- **Link oficial:** Engenharia de Software Moderna — Capítulo 2: Processos Ágeis (Seção sobre Scrum)
- **Scrum.org (Glossário e Guia de Papéis):**
- *Descrição:* Organização internacional de referência e certificação em Scrum, detalhando os papéis de Product Owner, Scrum Master e Developers.
- **Link oficial:** Scrum.org — What is a Scrum Team?

5- Eventos do Scrum

Eventos do Scrum

Tempo estimado de estudo: 10 a 15 minutos
Objetivo: entender a sequência dos eventos do Scrum, o propósito de cada um e como eles ajudam o time a inspecionar o trabalho e fazer ajustes durante a Sprint.
Imagem 1 — O ciclo da Sprint: use o infográfico que já criamos para mostrar a Sprint como o período que contém os eventos.
Imagem 2 — Os eventos na prática: use a ilustração com quatro cenas para acompanhar as explicações: planejamento, Daily Scrum, Review e Retrospective.

O que são os eventos do Scrum?

Os eventos do Scrum organizam momentos de trabalho em equipe. Eles ajudam o time a tornar o andamento do produto visível, verificar o que está acontecendo e adaptar o plano quando necessário.
O Guia do Scrum define a **Sprint** como um evento de duração fixa, de um mês ou menos, que contém os outros eventos. Quando uma Sprint termina, a próxima começa imediatamente. Por isso, a Sprint não é apenas uma reunião: é o ciclo inteiro em que o Scrum Team planeja, trabalha, verifica resultados e busca melhorar. Scrum Guides
Cada evento tem um propósito específico. Quando os eventos são conduzidos sem esse propósito, o time perde oportunidades de conversar sobre o trabalho e ajustar o caminho.

A sequência geral

```
Sprint Planning → trabalho da Sprint e Daily Scrums → Sprint Review →
Sprint Retrospective → próxima Sprint
```

As **Daily Scrums** acontecem ao longo da Sprint. A Review e a Retrospective ocorrem perto do encerramento do ciclo, nessa ordem.
Imagem sugerida: logo depois desta sequência, insira o infográfico do ciclo da Sprint. Ele oferece uma visão rápida antes de o estudante ler os detalhes de cada evento.

1. Sprint Planning — Planejamento da Sprint

A Sprint Planning abre a Sprint. Nela, o Scrum Team conversa sobre o valor esperado, o trabalho que pode ser realizado e a forma de começar esse trabalho.
O planejamento ajuda a responder a três perguntas:

- **Por que esta Sprint é valiosa?** A conversa ajuda a definir a **Sprint Goal**, ou Meta da Sprint.
- **O que pode ser feito nesta Sprint?** O time seleciona itens do Product Backlog que considera possível realizar.
- **Como o trabalho será feito?** Os Developers planejam como transformar os itens selecionados em um Incremento.

A Sprint Goal dá direção ao trabalho. Ela comunica o resultado que o time pretende alcançar, mesmo que detalhes do plano possam ser ajustados ao longo da Sprint.
Exemplo: para uma Sprint do Portal de Certificação, uma Sprint Goal poderia ser:
"Permitir que o candidato crie uma conta e acesse o portal com CPF e senha."
O Scrum Team poderia então selecionar itens relacionados ao cadastro e ao login e conversar sobre as etapas necessárias para realizá-los.

Duração

Para uma Sprint de um mês, a Sprint Planning tem duração máxima de oito horas. Em Sprints menores, tende a durar menos. Esse tempo é um limite máximo, não uma duração que o time precisa preencher por completo. scrumguides.org
Imagem sugerida: um cartão visual com três perguntas — "Por quê?", "O quê?" e "Como?" — ao lado de uma equipe organizando itens de trabalho.
Texto alternativo: "Equipe define a Meta da Sprint, seleciona itens do Product Backlog e planeja como realizar o trabalho."

2. Daily Scrum

A Daily Scrum é um evento diário, com duração máxima de 15 minutos. Os Developers a utilizam para inspecionar o progresso em direção à Sprint Goal e adaptar o plano de trabalho, se necessário.
O foco está no trabalho e no objetivo compartilhado. O time pode conversar sobre o que avançou, o que precisa ser reorganizado e se há algum ponto que exige colaboração adicional.
O Guia do Scrum não exige que todos respondam a uma lista fixa de perguntas. Os Developers podem escolher a estrutura que funciona melhor para eles, desde que a conversa ajude a

acompanhar o progresso rumo à Sprint Goal e gere um plano útil para o próximo período de trabalho. Scrum Guides
Exemplo: durante uma Sprint para implementar login, os Developers percebem que a validação do CPF está levando mais tempo do que esperavam. O plano pode ser adaptado para que uma pessoa ajude nessa parte, sem perder de vista a Meta da Sprint.

O que a Daily Scrum não é

A Daily Scrum não deve virar uma apresentação de tarefas para um chefe ou uma reunião em que cada pessoa fala apenas do próprio trabalho. Seu propósito é ajudar os Developers a coordenar o trabalho e decidir os próximos passos.

Duração

A Daily Scrum tem duração máxima de 15 minutos. O evento acontece todos os dias da Sprint. scrumguides.org
Imagem sugerida: um pequeno quadro de trabalho com cartões em andamento e o time conversando diante dele.
Texto alternativo: "Developers inspecionam o quadro da Sprint e combinam como avançar em direção à Meta da Sprint."

3. Sprint Review — Revisão da Sprint

A Sprint Review acontece perto do fim da Sprint. O Scrum Team e os principais stakeholders analisam o resultado do trabalho e conversam sobre os próximos passos do produto.
A conversa pode considerar:

- o que foi realizado;
- o que mudou no contexto ou nas necessidades do produto;
- quais aprendizados surgiram;
- quais oportunidades podem ser consideradas a seguir.

A Sprint Review é uma sessão de trabalho. Ela não deve ser reduzida a uma apresentação em que o time mostra telas enquanto os demais apenas assistem. Os participantes colaboram e podem usar o que aprenderam para adaptar os próximos passos, inclusive atualizando o Product Backlog. scrumguides.org
Exemplo: na Review do Portal, o time demonstra o fluxo de cadastro e login. Os stakeholders percebem que o candidato precisa receber uma mensagem clara quando o CPF informado já está cadastrado. Essa observação pode influenciar o trabalho futuro no produto.

Review não é o mesmo que aprovar ou reprovar pessoas

A Sprint Review não serve para avaliar individualmente quem trabalhou mais ou menos. O foco é inspecionar o produto, discutir o que foi aprendido e colaborar sobre o caminho seguinte.

Duração

Para uma Sprint de um mês, a Sprint Review tem duração máxima de quatro horas. Em Sprints menores, costuma durar menos. scrumguides.org

Imagem sugerida: equipe apresenta uma funcionalidade em um notebook e recebe comentários de outras pessoas.
Texto alternativo: "O Scrum Team demonstra um resultado funcional e conversa com stakeholders sobre o que fazer a seguir."

4. Sprint Retrospective — Retrospectiva da Sprint

Depois da Sprint Review, o Scrum Team realiza a Sprint Retrospective. O objetivo é refletir sobre como o trabalho aconteceu e planejar maneiras de melhorar a qualidade e a eficácia do time.
O time pode conversar sobre aspectos como:

- comunicação e colaboração;
- processos e ferramentas;
- qualidade do trabalho;
- dificuldades encontradas;
- práticas que ajudaram ou atrapalharam.

A conversa deve resultar em melhorias úteis que o time possa colocar em prática. Não é necessário tentar corrigir tudo de uma vez: é melhor escolher mudanças possíveis e acompanhar se elas ajudam.
Exemplo: durante uma Sprint do Portal, o time percebe que algumas dúvidas sobre os critérios das tarefas só aparecem perto do fim. Como melhoria, pode combinar que os itens sejam discutidos com mais antecedência no refinamento do Product Backlog.

Retrospective não é uma busca por culpados

A Retrospective deve ajudar o time a aprender e melhorar. O objetivo não é apontar uma pessoa como causa dos problemas, mas compreender o que aconteceu e decidir como trabalhar melhor.

Duração

Para uma Sprint de um mês, a Sprint Retrospective tem duração máxima de três horas. Ela conclui a Sprint; depois dela, inicia-se a próxima Sprint. scrumguides.org
Imagem sugerida: equipe organiza cartões em duas áreas: "O que funcionou?" e "O que podemos melhorar?".
Texto alternativo: "O time reflete sobre a Sprint e escolhe uma melhoria para experimentar no próximo ciclo."

Limites de tempo dos eventos

Os tempos abaixo são **limites máximos para uma Sprint de um mês**. Em Sprints menores, os eventos costumam ser mais curtos.

Evento	Duração máxima
Sprint Planning	8 horas
Daily Scrum	15 minutos
Sprint Review	4 horas
Sprint Retrospective	3 horas

A Sprint também tem duração fixa: pode durar até um mês. O limite de tempo ajuda a manter o foco do evento, mas não significa que o time precise usar todo o período disponível. scrumguides.org

Como os eventos se complementam

Cada evento ajuda a responder a uma pergunta diferente:

Evento	Pergunta para lembrar
Sprint Planning	Qual é o objetivo da Sprint, o que pode ser feito e como começar?
Daily Scrum	Como está o progresso rumo à Sprint Goal e o que ajustar no plano?
Sprint Review	O que foi produzido e o que aprendemos sobre os próximos passos do produto?
Sprint Retrospective	Como podemos melhorar a forma de trabalhar na próxima Sprint?

Em conjunto, os eventos formam um ciclo de planejamento, acompanhamento, avaliação do resultado e melhoria. O plano pode ser adaptado conforme o time aprende, mantendo a Sprint Goal como referência.

Exemplo completo: uma Sprint do Portal

Imagine que o Scrum Team tenha definido a Meta da Sprint: **"Permitir que o candidato crie uma conta e entre no portal."**

1. **Sprint Planning:** o time conversa sobre o valor dessa Meta, seleciona itens relacionados ao cadastro e ao login e planeja como começar.
2. **Daily Scrum:** durante a Sprint, os Developers verificam o progresso e adaptam o plano diante de descobertas ou dificuldades.
3. **Sprint Review:** o time demonstra o resultado e conversa com stakeholders sobre o que foi entregue e o que pode ser melhorado no produto.
4. **Sprint Retrospective:** o Scrum Team reflete sobre como trabalhou e escolhe uma melhoria para experimentar na Sprint seguinte.

Esse exemplo é apenas uma forma de visualizar os eventos. Em um projeto real, o conteúdo da Sprint depende da Meta da Sprint, do Product Backlog e do contexto do time.

Confira seu entendimento

1. Qual evento inicia a Sprint?
Resposta: Sprint Planning.
2. Qual evento ajuda os Developers a inspecionar diariamente o progresso rumo à Sprint Goal?
Resposta: Daily Scrum.
3. Em qual evento o Scrum Team colabora com stakeholders para inspecionar o resultado e discutir os próximos passos do produto?
Resposta: Sprint Review.
4. Qual evento ajuda o Scrum Team a planejar melhorias na forma de trabalhar?
Resposta: Sprint Retrospective.

5. Uma pessoa diz: "A Daily é o momento de cada desenvolvedor provar ao Scrum Master que trabalhou." Essa descrição está correta?
Resposta: Não. O foco da Daily Scrum é inspecionar o progresso rumo à Sprint Goal e adaptar o plano de trabalho.
6. Uma equipe usa a Sprint Review apenas para mostrar telas prontas, sem conversar sobre o produto ou seus próximos passos. O propósito do evento está sendo bem aproveitado?
Resposta: Não. A Review é uma sessão de trabalho colaborativa, não apenas uma apresentação.

Resumo da aula

- A **Sprint** é o ciclo que contém os demais eventos e dura até um mês.
- A **Sprint Planning** inicia a Sprint e ajuda o time a definir a Meta da Sprint, o trabalho possível e como começá-lo.
- A **Daily Scrum** ocorre todos os dias para inspecionar o progresso e adaptar o plano.
- A **Sprint Review** permite inspecionar o resultado do produto com os stakeholders e discutir próximos passos.
- A **Sprint Retrospective** ajuda o Scrum Team a planejar melhorias na qualidade e na eficácia do trabalho.
- Os eventos têm propósitos diferentes, mas juntos ajudam a equipe a aprender e adaptar o trabalho ao longo do ciclo.

Sugestão de imagens para a página

1. **Linha do tempo da Sprint:** o infográfico criado anteriormente, logo após a introdução.
2. **Os quatro eventos em situações práticas:** a ilustração com quatro cenas, antes das explicações detalhadas.
3. **Planning:** equipe organizando cartões e definindo um objetivo.
4. **Daily Scrum:** Developers conversando diante do quadro da Sprint.
5. **Review:** demonstração do produto com participação de stakeholders.
6. **Retrospective:** equipe discutindo o que funcionou e escolhendo melhorias.

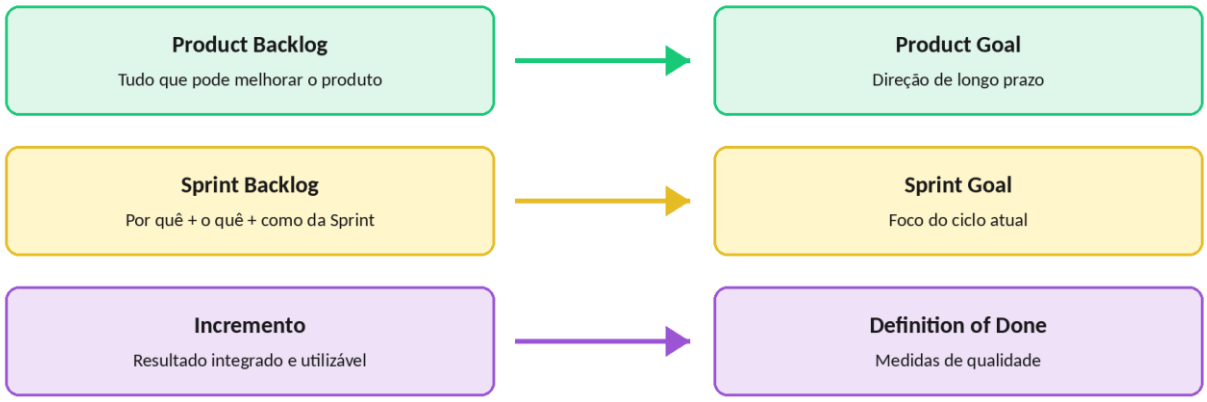
As imagens 3 a 6 podem ser usadas como pequenos cards ao lado de cada evento; a ilustração de quatro cenas também pode servir como visão geral antes do conteúdo. A base conceitual desta aula segue o Guia do Scrum 2020, disponível no site oficial do Scrum Guides

---



6- Artefatos do Scrum

Figura 1 - Relação entre os artefatos do Scrum e seus compromissos



Fonte: elaboração própria (2026), com base em Schwaber e Sutherland (2020).

Explicação da imagem

O diagrama mostra três pares inseparáveis. Cada artefato torna uma parte do trabalho ou do valor transparente; seu compromisso fornece o foco e a referência usados para medir o progresso. As setas representam associação, não uma sequência de etapas.

Introdução

Os artefatos do Scrum representam trabalho ou valor e existem para maximizar a transparência das informações essenciais. O Guia do Scrum define três artefatos formais: **Product Backlog**, **Sprint Backlog** e **Incremento**. Para cada um há um compromisso correspondente: **Product Goal**, **Sprint Goal** e **Definition of Done** (SCHWABER; SUTHERLAND, 2020).

Esses compromissos não são promessas de concluir uma quantidade fixa de tarefas. Eles orientam as decisões, aumentam a transparência e oferecem pontos concretos para inspeção e adaptação.

1. Product Backlog e Product Goal

O **Product Backlog** é uma lista ordenada e emergente do que é necessário para melhorar o produto. É a fonte única de trabalho realizado pelo Scrum Team. Como é emergente, muda à medida que o produto, o mercado e o entendimento das pessoas evoluem.

O **Product Goal** descreve um estado futuro do produto e funciona como objetivo de longo prazo do Scrum Team. O time deve cumprir ou abandonar um Product Goal antes de assumir outro (SCHWABER; SUTHERLAND, 2020).

Aplicação no portal

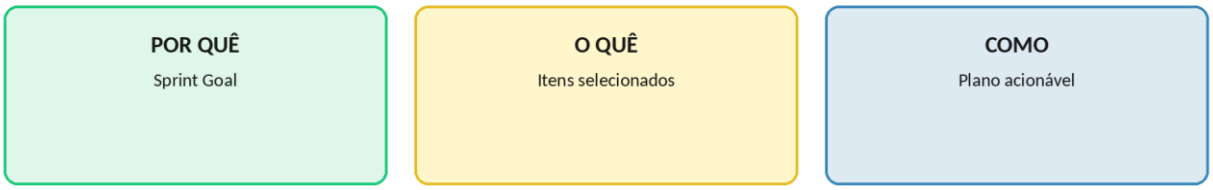
Elemento	Exemplo proposto
Product Goal	Permitir que candidatos estudem Scrum, verifiquem seu aprendizado e concluam uma certificação no portal.
Itens do Product Backlog	Cadastrar candidato; autenticar; consultar materiais por tema; responder questões; consultar resultado; emitir certificado.
Critério de ordenação	Valor para o candidato, riscos, dependências e conhecimento adquirido durante o desenvolvimento.

O Product Owner responde por maximizar o valor do produto e pela gestão eficaz do Product Backlog. Isso inclui comunicar o Product Goal, criar e ordenar itens e garantir que o backlog seja transparente e compreendido.

2. Sprint Backlog e Sprint Goal

O Sprint Backlog contém o Sprint Goal, os itens do Product Backlog selecionados para a Sprint e um plano acionável para criar o Incremento. Ele é criado pelos Developers e atualizado durante a Sprint conforme o aprendizado aumenta.

Figura 2 - Os três componentes do Sprint Backlog



Fonte: elaboração própria (2026), com base em Schwaber e Sutherland (2020).

O Sprint Goal é o objetivo único da Sprint. Ele cria coerência e foco, mas permite flexibilidade sobre o trabalho exato necessário para alcançá-lo. Caso o trabalho se revele diferente do esperado, os Developers colaboram com o Product Owner para negociar o escopo sem comprometer a meta.

Exemplo proposto para o portal

Sprint Goal: permitir que uma pessoa crie credenciais válidas e acesse o portal.

Itens selecionados: cadastro de candidato e autenticação por CPF e senha.

Plano inicial: validar dados; persistir o cadastro; proteger a senha; autenticar; tratar erros; testar o fluxo integrado.

3. Incremento e Definition of Done

Um Incremento é um passo concreto em direção ao Product Goal. Ele precisa somar-se aos incrementos anteriores, ser verificado e permanecer utilizável. Podem existir vários incrementos em uma Sprint, e valor pode ser liberado antes da Sprint Review.

A Definition of Done (DoD) descreve formalmente o estado do Incremento quando as medidas de qualidade exigidas para o produto foram atendidas. O trabalho que não satisfaz a DoD não faz parte do Incremento e retorna ao Product Backlog (SCHWABER; SUTHERLAND, 2020).

Definition of Done proposta para o portal

Dimensão	Condição para considerar o trabalho pronto
Funcional	Critérios de aceitação atendidos e fluxo principal validado de ponta a ponta.
Segurança	Validações executadas no servidor; senha armazenada como hash; dados sensíveis não expostos.
Dados	Restrições de integridade aplicadas; CPF duplicado tratado sem criar outra conta.
Qualidade	Testes relevantes aprovados; código revisado; erros compreensíveis e acessibilidade básica verificada.
Integração	Mudança integrada ao produto e utilizável no ambiente definido pelo time.

4. Como os três pares trabalham juntos

O Product Goal indica a direção estratégica. Em cada Sprint, o Sprint Goal define um resultado intermediário coerente. O Incremento oferece evidência concreta do progresso, desde que satisfaça a Definition of Done. A inspeção do que foi realmente concluído alimenta novas decisões sobre o Product Backlog.

Pontos importantes

O compromisso do Sprint Backlog é o Sprint Goal, e não a conclusão rígida de todos os itens inicialmente selecionados.

A Sprint Review inspeciona resultados e adapta o Product Backlog; ela não é uma barreira obrigatória para liberar valor.

Critérios de aceitação verificam um item específico. A Definition of Done estabelece a qualidade comum exigida para qualquer Incremento.

A DoD precisa ser respeitada por todos os Scrum Teams que trabalham no mesmo produto.

5. Síntese aplicada ao portal

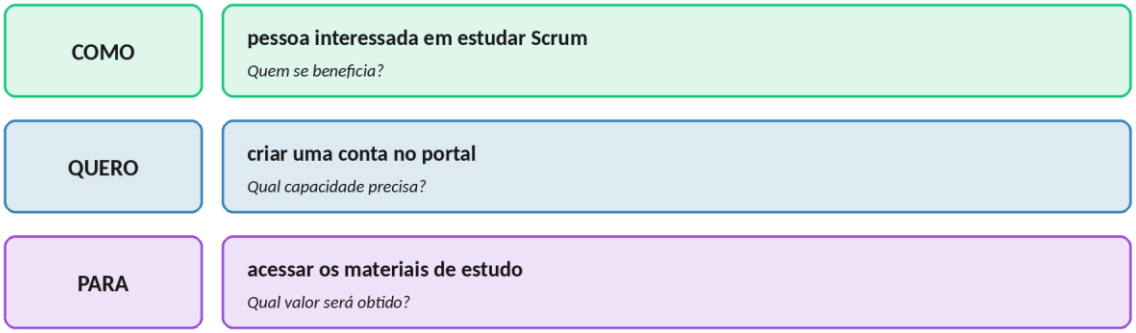
Artefato	Compromisso	Pergunta de inspeção
Product Backlog	Product Goal	O trabalho ordenado continua conduzindo ao objetivo do produto?
Sprint Backlog	Sprint Goal	O plano atual ainda oferece a melhor forma de alcançar a meta da Sprint?
Incremento	Definition of Done	O resultado está integrado, utilizável e atende às medidas de qualidade?

REFERÊNCIAS

SCHWABER, Ken; SUTHERLAND, Jeff. **The Scrum Guide: the definitive guide to Scrum: the rules of the game**. [S. l.]: Scrum Guides, 2020. Disponível em: https://scrumguides.org/scrum-guide.html. Acesso em: 19 out. 2026.

6- User Stories

Figura 1 - Anatomia de uma User Story aplicada ao portal



Fonte: elaboração própria (2026), com base em Agile Alliance ([s. d.]d).

Explicação da imagem

A figura separa os três elementos do modelo mais conhecido: perfil, capacidade e benefício. A história informa quem precisa de algo, o que deseja fazer e por que isso tem valor. O formato ajuda a organizar a conversa, mas não substitui os detalhes descobertos pelo time.

Introdução

Uma User Story é uma forma breve de representar uma necessidade pela perspectiva de quem usa ou se beneficia do produto. Em práticas ágeis, histórias ajudam a organizar o trabalho em incrementos funcionais que contribuem para o valor do produto (AGILE ALLIANCE, [s. d.]c).

User Stories não são um elemento obrigatório do Scrum. Elas são uma técnica possível para escrever itens do Product Backlog. Uma história útil permanece aberta à conversa, pode ser testada e expressa valor observável, em vez de apenas uma tarefa técnica.

1. Modelo 'Como... Quero... Para...'

O modelo associa três perguntas essenciais:

Como [perfil]: quem tem a necessidade ou recebe o benefício?

Quero [capacidade]: o que essa pessoa precisa conseguir fazer?

Para [benefício]: que resultado ou valor espera alcançar?

O modelo ficou conhecido como formato de papel-capacidade-benefício. A frase deve ser pequena o suficiente para funcionar como lembrete da conversa, não como uma especificação completa (AGILE ALLIANCE, [s. d.]d).

2. História vaga e história clara

Versão	História	Análise
Vaga	Como usuário, quero um cadastro bom para usar o sistema.	'Usuário' é genérico; 'bom' não é verificável; o benefício não esclarece o resultado esperado.
Clara	Como pessoa interessada em estudar Scrum, quero criar uma conta no portal para acessar os materiais de estudo.	Identifica o perfil, apresenta uma capacidade única e torna o benefício compreensível.

Mesmo a versão clara precisa ser discutida. O time deve esclarecer os dados necessários, regras de validação, estados de erro, limites de segurança e comportamento após o sucesso.

3. Três C: cartão, conversa e confirmação

Cartão: registra uma descrição breve da necessidade e do valor esperado.

Conversa: reúne Product Owner, Developers e pessoas relevantes para descobrir detalhes, exemplos e alternativas.

Confirmação: define exemplos ou critérios observáveis que demonstram se a história foi implementada corretamente.

Essa leitura evita transformar a história em um contrato fechado. Os detalhes evoluem conforme a história se aproxima da implementação e o time aprende mais sobre o problema.

4. Critérios de aceitação

Critérios de aceitação estabelecem condições observáveis para aceitar uma história. Eles delimitam o comportamento esperado e ajudam produto, desenvolvimento e testes a compartilhar o mesmo entendimento. Uma forma comum de escrevê-los é Dado - Quando - Então (AGILE ALLIANCE, [s. d.]a).

História de cadastro

US-01: Como pessoa interessada em estudar Scrum, quero criar uma conta com nome completo, CPF, e-mail e senha para obter credenciais de acesso ao portal.

Critério	Dado	Quando	Então
CA01 - Sucesso	CPF válido ainda não cadastrado, demais dados válidos e termos aceitos	o formulário é enviado	a conta é criada, o sucesso é informado e a pessoa é direcionada ao login
CA02 - Dados inválidos	nome sem sobrenome, CPF inválido ou e-mail malformado	há tentativa de envio	o erro é indicado e o cadastro não é concluído
CA03 - Senha	senha com menos de oito caracteres ou sem letras e números	há tentativa de cadastro	o envio é impedido com mensagem explicativa
CA04 - Duplicidade	CPF já cadastrado	uma nova conta é solicitada	outra conta não é criada e a duplicidade é informada
CA05 - Aceite	termos não aceitos	há tentativa de cadastro	o envio é impedido; no sucesso, a data de aceite é registrada

Os critérios acima foram definidos para o cenário de cadastro do portal. Eles funcionam como especificação didática e devem ser validados pelo time durante o refinamento e a implementação.

5. Critérios de aceitação e Definition of Done

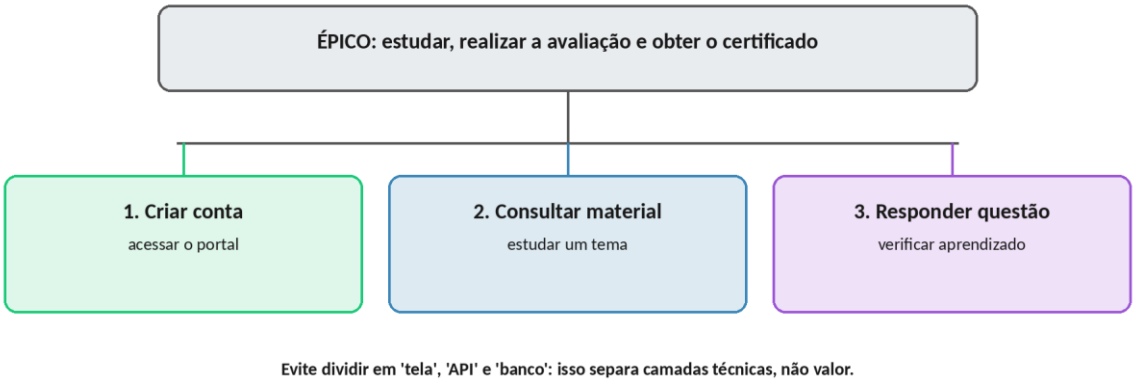
Critérios de aceitação	Definition of Done
Específicos de uma história ou item.	Comum aos incrementos do produto.
Descrevem comportamentos e resultados esperados.	Estabelece as medidas de qualidade necessárias.
Exemplo: impedir cadastro com CPF inválido.	Exemplo: testes aprovados, senha protegida e mudança integrada.

Atender aos critérios de aceitação não dispensa a DoD. Uma história pode produzir o comportamento correto e ainda não estar pronta caso falhe nas medidas comuns de qualidade, segurança ou integração.

6. Como dividir histórias grandes

Uma história deve ser pequena o suficiente para ser concluída dentro de uma iteração. Dividir consiste em criar histórias menores preservando valor mensurável em cada parte (AGILE ALLIANCE, [s. d.]b).

Figura 2 - Divisão vertical de uma história grande em resultados menores



Fonte: elaboração própria (2026), com base em Agile Alliance ([s. d.]b).

A divisão vertical atravessa as camadas necessárias para entregar uma capacidade utilizável. Uma divisão horizontal, como 'criar tela', 'criar API' e 'criar tabela', gera componentes isolados que não oferecem valor observável ao candidato.

Técnicas úteis de divisão

Etapas do fluxo: cadastrar-se; entrar; consultar um material; responder uma questão; consultar resultado.

Cenários: primeiro o caminho de sucesso; depois duplicidade, recuperação e exceções relevantes.

Regras de negócio: iniciar com a regra essencial e adicionar regras menos frequentes em histórias posteriores.

Operações: separar criar, consultar, atualizar ou cancelar quando cada operação possui valor próprio.

Dados ou variações: começar com um tipo, perfil ou canal e ampliar conforme a prioridade.

7. Verificação de qualidade da história

A heurística INVEST ajuda o time a avaliar histórias: independente, negociável, valiosa, estimável, pequena e testável. Não é uma certificação automática; é um conjunto de perguntas para orientar o refinamento (AGILE ALLIANCE, [s. d.]c).

Critério	Pergunta de verificação
Independente	A ordem de implementação pode mudar sem quebrar a história?
Negociável	Existe espaço para conversar sobre a melhor solução?
Valiosa	O resultado melhora algo para uma pessoa ou para o produto?
Estimável	O time entende o suficiente para estimar seu tamanho relativo?
Pequena	Pode ser concluída dentro da Sprint com folga para inspeção?
Testável	Há critérios observáveis que confirmam o comportamento?

8. Exemplo completo para o portal

Título: Criar conta de candidato

História: Como pessoa interessada em estudar Scrum, quero criar uma conta com meus dados para obter credenciais de acesso ao portal.

Valor: habilitar o primeiro acesso à plataforma.

Fora do escopo: autenticação, recuperação de senha e emissão de certificado.

Critérios: CA01 a CA05 apresentados anteriormente.

Dependências: persistência de candidatos e disponibilização dos termos de uso.

Qualidade: atender à Definition of Done do produto.

REFERÊNCIAS

AGILE ALLIANCE. **Given-When-Then**. [S. l.], [s. d.]a. Disponível em: https://agilealliance.org/glossary/given-when-then/. Acesso em: 19 out. 2026.

AGILE ALLIANCE. **Story splitting**. [S. l.], [s. d.]b. Disponível em: https://agilealliance.org/glossary/story-splitting/. Acesso em: 19 out. 2026.

AGILE ALLIANCE. **User stories**. [S. l.], [s. d.]c. Disponível em: https://agilealliance.org/glossary/user-stories/. Acesso em: 19 out. 2026.

AGILE ALLIANCE. **User story template**. [S. l.], [s. d.]d. Disponível em: https://agilealliance.org/glossary/user-story-template/. Acesso em: 19 out. 2026.

SCHWABER, Ken; SUTHERLAND, Jeff. **The Scrum Guide: the definitive guide to Scrum: the rules of the game**. [S. l.]: Scrum Guides, 2020. Disponível em: https://scrumguides.org/scrum-guide.html. Acesso em: 19 out. 2026.

7- Gestão do Product Backlog



Explicação da Imagem

Este diagrama simplificado apresenta uma visualização direta e prática de como as demandas reais de um portal de estudos são organizadas no **Product Backlog**:

- **Topo (Alta Prioridade):** Contém a funcionalidade essencial e pré-requisito técnico para o funcionamento de qualquer portal, como a **Tela de Login e Cadastro**, que deve ser desenvolvida primeiro.
- **Meio (Média Prioridade):** Traz o núcleo do produto que entrega valor imediato ao usuário logo após o acesso, como a **Listagem de Cursos**.

- **Base (Baixa Prioridade / Futuro):** Agrupa funcionalidades importantes, mas que dependem da base estar pronta, como o **Sistema de Pagamento por Pix**.

Introdução

Imagine que você está construindo uma casa. Você tem uma lista enorme de desejos: colocar a piscina, erguer as paredes, escolher a cor dos azulejos do banheiro e instalar o telhado. Se você tentar fazer tudo ao mesmo tempo, a casa desaba. No desenvolvimento de software, o **Product Backlog** é exatamente essa grande lista viva e organizada de tudo o que é necessário para construir ou melhorar um produto.

No contexto de metodologias ágeis (como o Scrum), o Product Backlog é a única fonte de requisitos para qualquer mudança a ser feita no produto. Ele nunca está completo; ele evolui à medida que o produto e o ambiente em que ele é usado mudam.

Conceitos Principais

- **Product Owner (PO):** É o único responsável pelo Product Backlog. Cabe a ele gerenciar a lista, garantir que ela esteja visível, transparente e clara para todos, além de definir a ordem (priorização) dos itens para entregar o máximo de valor ao negócio e aos usuários.

- **Colaboração do Time:** Embora o PO decida o *o quê* (a ordem e o valor), o time de desenvolvimento colabora ativamente estimando o esforço necessário (**Story Points**) e refinando os detalhes técnicos.

- **Refinamento (Backlog Refinement):** É o ato contínuo de detalhar, estimar e reavaliar os itens do backlog para garantir que as histórias de usuário estejam prontas para entrar nas próximas Sprints.

Em resumo

• O Product Backlog é uma lista dinâmica e priorizada de tudo que pode ser desenvolvido em um produto.
• É responsabilidade do Product Owner (PO) manter o backlog atualizado, ordenado e transparente.

• Principais conceitos:

• Ordenação e priorização: os itens mais valiosos para o negócio e usuários ficam no topo.
• Refinamento: épicos e funcionalidades são detalhados em histórias de usuário com critérios claros.

• Colaboração: o PO decide a ordem, mas o time contribui com estimativas e insights técnicos.

• Exemplo: em um portal de serviços, funcionalidades como “login seguro” e “busca rápida” aparecem no topo, pois são críticas para a experiência inicial do usuário.

Fontes consultadas

• Scrum.com.br – Guia de Product Backlog scrum.com.br
• Escritório de Projetos – Product Backlog no Scrum Escritório d...
• Scrum Alliance – Boas práticas de backlog rework.com

8- Kanban



Este diagrama minimalista e estruturado segue o modelo clássico de fluxo de trabalho:

O quadro é dividido em quatro etapas sequenciais:

- **1- A Fazer:** Onde ficam armazenadas as demandas iniciais, como *"Criar Tela de Login"* e *"Listar Cursos"*.
- **2- Em Andamento:** O espaço reservado para os itens que estão sendo executados ativamente pela equipe.
- **3- Concluído:** Onde as tarefas finalizadas aguardam validação.
- **4- Aprovado(opcional):** A etapa final onde o trabalho é aceito e integrado ao portal.

1. Kanban: Fluxo Contínuo e Eficiência

Introdução

Se o Scrum organiza o trabalho em ciclos (as Sprints), o **Kanban** foca em **como o trabalho flui** de ponta a ponta no dia a dia. Originado na indústria automobilística (sistema Toyota de produção), o Kanban utiliza a visualização gráfica para gerenciar gargalos, reduzir desperdícios e entregar valor de forma fluida e sustentável.

Conceitos Principais

O Kanban apoia-se em princípios fundamentais para otimizar qualquer processo de desenvolvimento ou operação:

- **Visualização do Fluxo de Trabalho:** O uso de um quadro (físico ou digital) com colunas que representam o ciclo de vida da tarefa (ex: *A Fazer, Em Desenvolvimento, Em Teste, Concluído*). Todos enxergam onde cada item está.

- **Limitação do WIP (*Work in Progress*):** Significa limitar a quantidade de tarefas que podem estar em andamento ao mesmo tempo. Multitarefa diminui a produtividade; focar em terminar antes de começar acelera a entrega.

- **Gestão de Gargalos:** Identificar pontos onde o trabalho se acumula (filas) e agir sobre eles para desbloquear o fluxo.

- **Melhoria Contínua (Kaizen):** Ajustes incrementais e evolutivos baseados em métricas de tempo de ciclo (*cycle time*) e inspeções regulares.

Em resumo

• O Kanban é um método ágil que visualiza o fluxo de trabalho em colunas e cartões.

• **Conceitos principais**:

• Fluxo contínuo: tarefas avançam sem ciclos fixos, sempre puxadas conforme capacidade.

• Limite de WIP (Work in Progress): cada coluna tem um número máximo de itens, evitando sobrecarga.

• Gargalos: quando uma coluna atinge o limite, o time foca em destravar antes de iniciar novas tarefas.

. Fontes Consultadas

- **Kanban Guide** – David J. Anderson e Lean-Kanban University. Disponível em: kanban.university.

- **Kanban: Successful Evolutionary Change for Your Technology Business** – David J. Anderson (O livro fundamental sobre o método).

- **Agile Project Management with Kanban** – Eric Brechner (Abordagem prática para equipes de software).

9- Planejamento Ágil

Planejamento Ágil

1. Introdução ao Planejamento Ágil

O **Planejamento Ágil** é uma abordagem de organização do trabalho que permite desenvolver produtos e projetos de maneira incremental, colaborativa e adaptável. Em vez de definir todos os detalhes do projeto no início e seguir um plano rígido, a equipe planeja o trabalho em ciclos curtos, entrega resultados frequentemente e ajusta suas prioridades de acordo com o feedback recebido.

Essa abordagem é muito utilizada no desenvolvimento de software, mas também pode ser aplicada em projetos de marketing, gestão de produtos, operações e outras áreas.

Os principais objetivos do planejamento ágil são:

- **Entregar valor continuamente:** priorizar funcionalidades que tragam benefícios reais ao cliente.
- **Adaptar-se às mudanças:** revisar planos conforme novas necessidades surgem.
- **Melhorar a previsibilidade:** utilizar dados das entregas anteriores para estimar o trabalho futuro.
- **Promover colaboração:** envolver as pessoas responsáveis pelo produto e pela execução.
- **Reduzir riscos:** identificar problemas antecipadamente e validar as entregas em etapas menores.

1.1. Planejamento tradicional versus planejamento ágil

Planejamento tradicional	**Planejamento ágil**
Busca definir grande parte do projeto antecipadamente.	Planeja continuamente, em diferentes níveis.
Mudanças podem exigir revisões extensas do plano.	Mudanças são avaliadas e incorporadas de forma controlada.

A entrega costuma ocorrer ao final de fases ou do projeto.	Entregas de valor podem ocorrer a cada iteração.
Estimativas frequentemente se baseiam em horas e prazos fixos.	Pode utilizar Story Points, velocidade e outras métricas.
O sucesso é frequentemente avaliado pelo cumprimento do plano.	Valor entregue, qualidade, resultados e capacidade de adaptação também são considerados.

É importante destacar que o planejamento ágil não significa ausência de planejamento. Significa planejar continuamente, utilizando informações atualizadas para tomar decisões melhores.

2.3. O que é uma Sprint?

Uma Sprint é um período de duração fixa de **um mês ou menos**, durante o qual a equipe trabalha para atingir um objetivo e produzir um incremento de valor.

É comum utilizar Sprints de uma, duas ou três semanas. A duração deve ser adequada ao contexto do produto e manter uma cadência consistente.

Uma Sprint pode envolver:

1. Sprint Planning — planejamento da Sprint.
2. Daily Scrum — acompanhamento diário do progresso pelos Developers.
3. Trabalho de desenvolvimento, testes e integração.
4. Sprint Review — inspeção do resultado e discussão de adaptações futuras.
5. Sprint Retrospective — identificação de melhorias no processo de trabalho.

A Sprint Planning inicia a Sprint, e a Sprint Retrospective a encerra. O trabalho de desenvolvimento acontece ao longo de todo o ciclo.

3. Sprint Planning: planejamento da Sprint

O **Sprint Planning** é o evento do Scrum que define por que a Sprint é importante, o que pode ser realizado e como o trabalho será executado.

É o momento em que o Scrum Team estabelece uma direção comum para a próxima Sprint. O evento não deve ser apenas uma distribuição de tarefas: ele precisa promover entendimento compartilhado sobre o objetivo e o trabalho necessário.

3.1. Objetivos do Sprint Planning

O planejamento da Sprint procura responder a três perguntas essenciais:

- **Por que esta Sprint é valiosa?** Define-se o Sprint Goal.

- **O que pode ser concluído nesta Sprint?** Selecionam-se os itens do Product Backlog.
- **Como o trabalho escolhido será realizado?** Desenvolve-se o plano de execução.

As três respostas devem estar conectadas. Selecionar muitas tarefas sem um objetivo claro pode resultar em uma Sprint ocupada, mas sem uma entrega coerente de valor.

3.2. Quem participa?

Participam do Sprint Planning o Product Owner, o Scrum Master e os Developers.

- O Product Owner apresenta as prioridades, o valor esperado e os itens mais relevantes do Product Backlog.
- Os Developers avaliam a capacidade disponível, esclarecem dúvidas, selecionam o trabalho que consideram viável e planejam sua execução.
- O Scrum Master ajuda a garantir que o evento seja compreendido e produtivo, facilitando o processo quando necessário.

O planejamento deve ser colaborativo. O Product Owner não deve simplesmente impor uma lista de tarefas, e a equipe não deve selecionar itens sem compreender o objetivo de negócio.

3.3. Como realizar um Sprint Planning passo a passo

Etapa 1 — Definir o objetivo da Sprint

Identificar o resultado que trará valor ao produto. O objetivo deve ser claro, compreensível e permitir alguma flexibilidade na escolha das soluções.

Etapa 2 — Analisar o Product Backlog

Revisar os itens prioritários, critérios de aceitação, dependências, riscos e dúvidas. Itens bem refinados facilitam a tomada de decisão.

Etapa 3 — Avaliar a capacidade da equipe

Considerar férias, feriados, reuniões, suporte, atividades paralelas, experiência da equipe e resultados de Sprints anteriores.

Etapa 4 — Selecionar os itens da Sprint

Os Developers selecionam, em colaboração com o Product Owner, os itens que podem ser concluídos considerando o objetivo e a capacidade disponível.

Etapa 5 — Criar o plano de execução

Identificar atividades técnicas, testes, integrações, dependências e riscos. Decompor os itens em tarefas pode ajudar, mas não precisa resultar em um plano imutável.

Etapa 6 — Confirmar o entendimento

Verificar se todos compreendem o Sprint Goal, os itens selecionados, a estratégia de execução e a Definition of Done.

3.4. Duração do Sprint Planning

No Scrum Guide, para uma Sprint de um mês, o Sprint Planning tem duração máxima de oito horas. Para Sprints menores, o evento normalmente é mais curto.

Por exemplo, uma equipe com Sprints de duas semanas pode realizar um planejamento de duas a quatro horas, dependendo da complexidade do produto, da preparação do backlog e da experiência do time. Esse intervalo é uma referência prática, não uma regra oficial do Scrum.

3.5. Exemplo prático de Sprint Planning

Imagine uma equipe que desenvolve um aplicativo de vendas. O objetivo é facilitar a compra de produtos.

O Product Backlog contém os seguintes itens:

Item	Funcionalidade	Prioridade
US01	Consultar catálogo de produtos	Alta
US02	Adicionar produto ao carrinho	Alta
US03	Calcular o frete	Média
US04	Aplicar cupom de desconto	Média
US05	Criar lista de desejos	Baixa

A equipe decide trabalhar no objetivo: **permitir que o usuário selecione produtos e prepare seu carrinho de compras**.

São selecionadas as histórias de catálogo e carrinho, desde que sejam viáveis e possam atender à Definition of Done. O cálculo do frete poderá ficar para uma Sprint posterior, caso não seja necessário para atingir o objetivo atual.

O resultado do planejamento deve conter:

- Um Sprint Goal claro.
- Os itens selecionados para a Sprint.
- Um plano inicial de implementação.
- Os riscos e as dependências identificados.
- O entendimento compartilhado sobre o que significa concluir o trabalho.

Ponto importante: o escopo pode ser renegociado durante a Sprint entre os Developers e o Product Owner, desde que isso não coloque o Sprint Goal em risco. A qualidade não deve ser reduzida para simular uma entrega concluída.

4. Story Points: estimativa relativa do esforço

Story Points são uma unidade relativa utilizada por equipes ágeis para estimar o tamanho de um item do Product Backlog, como uma User Story.

Em vez de tentar prever exatamente quantas horas serão necessárias, a equipe compara os itens e considera fatores como:

- **Esforço:** quantidade de trabalho envolvida.
- **Complexidade:** dificuldade técnica ou funcional.
- **Incerteza:** falta de informações, requisitos desconhecidos ou soluções ainda não validadas.
- **Risco:** possibilidade de encontrar obstáculos durante a execução.

Story Points não são uma medida universal. Cada equipe desenvolve uma referência própria para estimar o tamanho relativo dos itens.

4.1. Por que utilizar Story Points?

Em projetos de software, prever o tempo exato de uma atividade pode ser difícil, especialmente quando existem integrações, dependências ou requisitos pouco claros.

A estimativa relativa permite que a equipe compare histórias e discuta suas diferenças sem precisar transformar imediatamente cada estimativa em horas.

Por exemplo:

- Uma história simples pode receber 1 ponto.
- Uma história um pouco maior pode receber 2 pontos.
- Uma história com maior complexidade pode receber 5 pontos.
- Uma história muito complexa ou incerta pode receber 13 pontos.

Esses valores são ilustrativos. Uma história de 5 pontos não necessariamente levará cinco horas ou cinco dias para ser concluída.

4.2. Escala de Story Points

Muitas equipes utilizam a sequência de Fibonacci ou uma escala adaptada, como 1, 2, 3, 5, 8, 13 e 21.

1

Muito pequeno

Trabalho simples, conhecido e com poucas dependências.

2

Pequeno

Pouco esforço e baixa incerteza.

3

Moderado

Trabalho relativamente conhecido, com algumas etapas.

5

Médio

Mais etapas, integrações ou complexidade.

8

Grande

Maior esforço, dependências ou incertezas.

13

Muito grande

Item difícil de estimar; pode precisar ser dividido.

A escala de Fibonacci aumenta a distância entre os valores conforme o tamanho cresce. Isso representa a dificuldade de estimar com precisão itens maiores e mais incertos.

Uma equipe também pode utilizar escalas como 0, 1, 2, 3, 5, 8 e 13. O mais importante é manter uma referência consistente dentro do time.

4.3. Como estimar Story Points na prática

Considere as seguintes histórias de usuário de um aplicativo:

User Story	**Estimativa ilustrativa**	**Justificativa**
Exibir o nome e o preço de um produto	1	Funcionalidade simples e conhecida.
Filtrar produtos por categoria	3	Exige regras de filtragem e testes.
Implementar carrinho com atualização de quantidades	5	Envolve regras de negócio, estado e validações.
Integrar pagamentos com um provedor externo	8	Pode envolver segurança, integração, erros e homologação.

Criar uma solução de recomendação personalizada ainda não validada	13	Alta incerteza técnica e funcional.

Os números servem para demonstrar o raciocínio. Uma equipe real pode atribuir outros valores após discutir os requisitos, a arquitetura e sua experiência anterior.

4.4. Story Points não são horas

Essa é uma das distinções mais importantes do planejamento ágil.

Story Points	**Horas**
Estimam tamanho relativo.	Medem ou estimam duração do trabalho.
Consideram esforço, complexidade e incerteza.	Representam tempo de execução.
São interpretados com base nas referências da equipe.	Podem ser registrados em horas ou frações de hora.
Não devem ser convertidos automaticamente em tempo.	Dependem das condições reais de trabalho.

Uma equipe não deve afirmar que um ponto equivale a quatro horas de maneira universal. Mesmo que exista uma correlação histórica em determinado contexto, ela não constitui uma regra geral nem garante previsões futuras.

4.5. Benefícios e limitações dos Story Points

Benefícios:

- Facilitam comparações entre itens.
- Estimulam discussões sobre complexidade e riscos.
- Ajudam a observar a capacidade histórica da equipe.
- Contribuem para estimar o volume provável de trabalho futuro.

Limitações:

- Dependem do entendimento compartilhado da equipe.
- Podem variar quando a composição ou a experiência do time muda.
- Não garantem a conclusão de uma quantidade fixa de trabalho.
- Podem ser mal utilizados quando se tornam metas individuais de produtividade.

Story Points não devem ser usados para comparar diretamente o desempenho de equipes diferentes, porque cada equipe possui suas próprias referências e condições de trabalho.

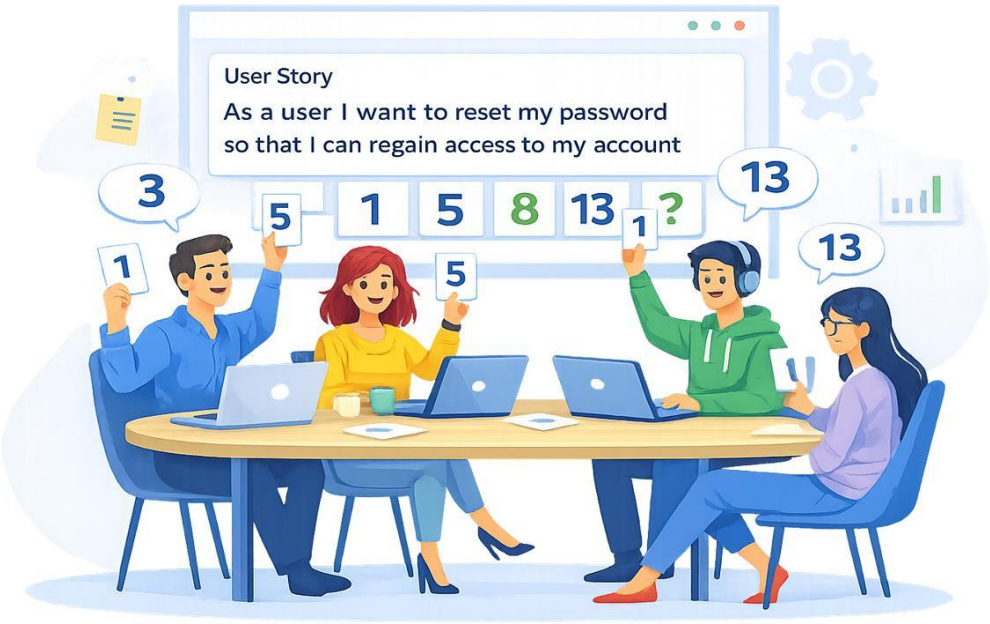
5. Planning Poker: técnica colaborativa de estimativa

O **Planning Poker** é uma técnica de estimativa coletiva usada para atribuir Story Points aos itens do Product Backlog. Ela combina julgamento individual, discussão em grupo e consenso para reduzir a influência de opiniões dominantes.

A técnica é especialmente útil quando os itens apresentam incertezas e precisam ser compreendidos por diferentes profissionais da equipe.







5.1. Como funciona o Planning Poker?

Cada participante recebe um conjunto de cartas com valores de estimativa, geralmente baseados na sequência de Fibonacci: 1, 2, 3, 5, 8, 13 e valores adicionais, conforme a escala escolhida.

O processo normalmente segue estas etapas:

1. **Apresentação da história:** o Product Owner ou responsável pelo refinamento explica a necessidade, os critérios de aceitação e o valor esperado.
2. **Esclarecimento de dúvidas:** os participantes discutem regras de negócio, restrições técnicas, dependências e riscos.
3. **Estimativa individual:** cada participante escolhe uma carta sem revelar sua estimativa aos demais.
4. **Revelação simultânea:** todos mostram suas cartas ao mesmo tempo, reduzindo a influência das primeiras opiniões apresentadas.
5. **Discussão das diferenças:** participantes com estimativas mais altas e mais baixas explicam seu raciocínio.
6. **Nova rodada:** depois de esclarecer as diferenças, a equipe realiza outra estimativa, se necessário.
7. **Registro da estimativa:** a equipe registra um valor consensual ou uma decisão suficientemente compartilhada para prosseguir.

O Planning Poker não exige unanimidade matemática em todos os casos. O objetivo é desenvolver entendimento comum sobre o tamanho relativo do trabalho.

5.2. Exemplo prático de Planning Poker

Uma equipe precisa estimar a história:

Como cliente, quero adicionar produtos ao carrinho para poder comprá-los posteriormente.

Na primeira rodada, quatro participantes apresentam as seguintes estimativas:

Desenvolvedor A

3

Desenvolvedor B

8

Profissional de testes

5

Desenvolvedor C

5

As estimativas variam de 3 a 8 pontos. Em vez de calcular simplesmente a média, a equipe investiga as razões dessa diferença.

- Quem estimou 3 pontos acredita que o carrinho utilizará componentes já existentes.
- Quem estimou 8 pontos identificou possíveis problemas de persistência, atualização de quantidades e integração com o estoque.
- Quem estimou 5 pontos considera que o escopo é viável, mas exige validações adicionais.

Após a discussão, a equipe percebe que a persistência do carrinho ainda não está definida. O grupo esclarece o requisito e realiza uma nova rodada. Supondo que haja consenso em 5 pontos, esse será o valor registrado.

Aprendizado: a maior contribuição do Planning Poker não é o número final, mas a descoberta de informações, riscos e diferentes interpretações da história.

5.3. Vantagens do Planning Poker

- Reduz a influência de participantes mais dominantes.
- Incentiva a participação de profissionais com diferentes especialidades.
- Ajuda a identificar requisitos incompletos.
- Promove conhecimento compartilhado sobre o produto.
- Permite discutir riscos antes de assumir compromissos de trabalho.

5.4. Boas práticas

- Refinar os itens antes da reunião de estimativa.

- Explicar os critérios de aceitação com clareza.
- Evitar que a estimativa de uma pessoa influencie as demais antes da revelação.
- Investigar diferenças significativas, sem discutir apenas para defender um número.
- Dividir histórias excessivamente grandes ou incertas.
- Não usar os pontos para pressionar a equipe a produzir mais.

O Planning Poker pode ser realizado presencialmente com cartas físicas ou remotamente com ferramentas de colaboração.

6. Planejamento de Sprint

O planejamento de Sprint é o processo mais amplo de preparação e organização do trabalho que será realizado durante uma iteração. No Scrum, o evento formal que inicia essa etapa é o Sprint Planning.

Para fins de estudo, é útil diferenciar o evento de planejamento do trabalho contínuo de preparação que o antecede.

6.1. O que deve estar preparado antes da reunião?

A equipe pode realizar atividades de refinamento do Product Backlog para facilitar o Sprint Planning:

- Entender as necessidades dos usuários.
- Revisar e ordenar os itens prioritários.
- Definir critérios de aceitação.
- Identificar dependências e riscos.
- Dividir itens grandes em histórias menores.
- Estimar os itens, quando isso for útil para a tomada de decisão.

O refinamento do Product Backlog é uma atividade contínua, não um evento formal do Scrum com duração fixa.

6.2. Capacidade da equipe e seleção do trabalho

A capacidade representa as condições reais que a equipe terá para trabalhar durante a Sprint. Não significa simplesmente multiplicar o número de pessoas pelos dias do período.

Considere uma equipe com cinco profissionais e uma Sprint de dez dias úteis. O total teórico é de 50 pessoas-dias, mas férias, feriados, suporte, reuniões e outras responsabilidades reduzem o tempo disponível para o trabalho planejado.

Por isso, a equipe deve combinar a avaliação de capacidade com o histórico de entrega e a complexidade dos itens.

6.3. Exemplo de planejamento com velocidade

Imagine que uma equipe concluiu os seguintes volumes de trabalho em três Sprints anteriores:

Sprint	Story Points concluídos
Sprint 1	20
Sprint 2	24
Sprint 3	22

A velocidade média histórica é:

20+24+223=22320+24+22=22

A equipe concluiu, em média, 22 pontos por Sprint.

Esse valor pode servir como referência inicial para discutir a seleção de trabalho na próxima Sprint, mas não representa uma garantia. A equipe ainda precisa considerar sua capacidade atual, as dependências, o Sprint Goal e o contexto dos itens.

Se houver férias, mudanças na equipe ou uma demanda extraordinária de suporte, pode ser adequado selecionar menos trabalho.

6.4. Sprint Goal: o objetivo da Sprint

O Sprint Goal descreve o resultado que a equipe pretende alcançar durante a Sprint. Ele orienta as decisões e mantém a equipe focada em um propósito comum.

Exemplo pouco adequado:

Desenvolver as histórias 1, 2 e 3.

Esse enunciado apenas enumera atividades.

Exemplo mais adequado:

Permitir que novos clientes encontrem produtos e adicionem itens ao carrinho.

Esse objetivo explica o resultado pretendido e permite que a equipe avalie como o trabalho contribui para ele.

6.5. O que acontece durante a Sprint?

Após o planejamento:

- Os Developers executam o trabalho e acompanham o progresso diariamente.
- A equipe inspeciona os resultados e adapta o plano conforme necessário.
- O Product Owner esclarece dúvidas e colabora na negociação do escopo.
- A qualidade é mantida conforme a Definition of Done.

- O Sprint Backlog é atualizado conforme a equipe aprende mais sobre o trabalho.

A Daily Scrum não é uma reunião de prestação de contas ao gerente. É um evento para os Developers inspecionarem o progresso em direção ao Sprint Goal e adaptarem o plano de trabalho.

6.6. Erros comuns no planejamento de Sprint

- Selecionar mais trabalho do que a equipe consegue concluir.
- Confundir estar ocupado com entregar valor.
- Iniciar histórias sem entendimento suficiente dos requisitos.
- Ignorar testes, integração, documentação necessária e critérios de conclusão.
- Transformar Story Points em metas individuais.
- Tratar o Sprint Backlog como um plano imutável.
- Priorizar a quantidade de histórias em vez do resultado da Sprint.

7. Planejamento de Releases

Uma **Release** é a disponibilização de uma versão do produto ou de funcionalidades para usuários, clientes ou outro público definido. O planejamento de releases procura coordenar as entregas ao longo de várias Sprints para alcançar objetivos de negócio.

Diferentemente do Sprint Planning, que organiza o trabalho de uma Sprint específica, o planejamento de releases observa um horizonte mais amplo.

Uma release pode envolver várias Sprints, e uma equipe também pode disponibilizar incrementos menores continuamente, sem precisar esperar o final de uma release formal.

7.1. Objetivos do planejamento de releases

- Definir quais funcionalidades são importantes para uma versão.
- Estabelecer metas e resultados de negócio.
- Estimar quando determinados resultados poderão estar disponíveis.
- Coordenar dependências entre equipes, sistemas e fornecedores.
- Identificar riscos técnicos e operacionais.
- Planejar testes de aceitação, segurança, implantação e comunicação.
- Alinhar expectativas de stakeholders e clientes.

O planejamento de releases deve equilibrar valor, risco, qualidade e previsibilidade. Uma data estimada não deve ser tratada como garantia quando existem incertezas significativas.

7.2. Elementos de um plano de release

Elemento	Descrição
Objetivo da release	Resultado de negócio que se deseja alcançar.
Escopo prioritário	Funcionalidades necessárias para alcançar o objetivo.
Backlog priorizado	Itens organizados segundo valor, risco e dependências.
Estimativas	Tamanho relativo dos itens e capacidade histórica.
Cronograma indicativo	Horizonte provável para a entrega.
Dependências	Integrações, fornecedores, aprovações ou outras equipes.
Critérios de qualidade	Testes, segurança, desempenho e Definition of Done.
Estratégia de implantação	Como disponibilizar a versão, monitorar resultados e responder a problemas.

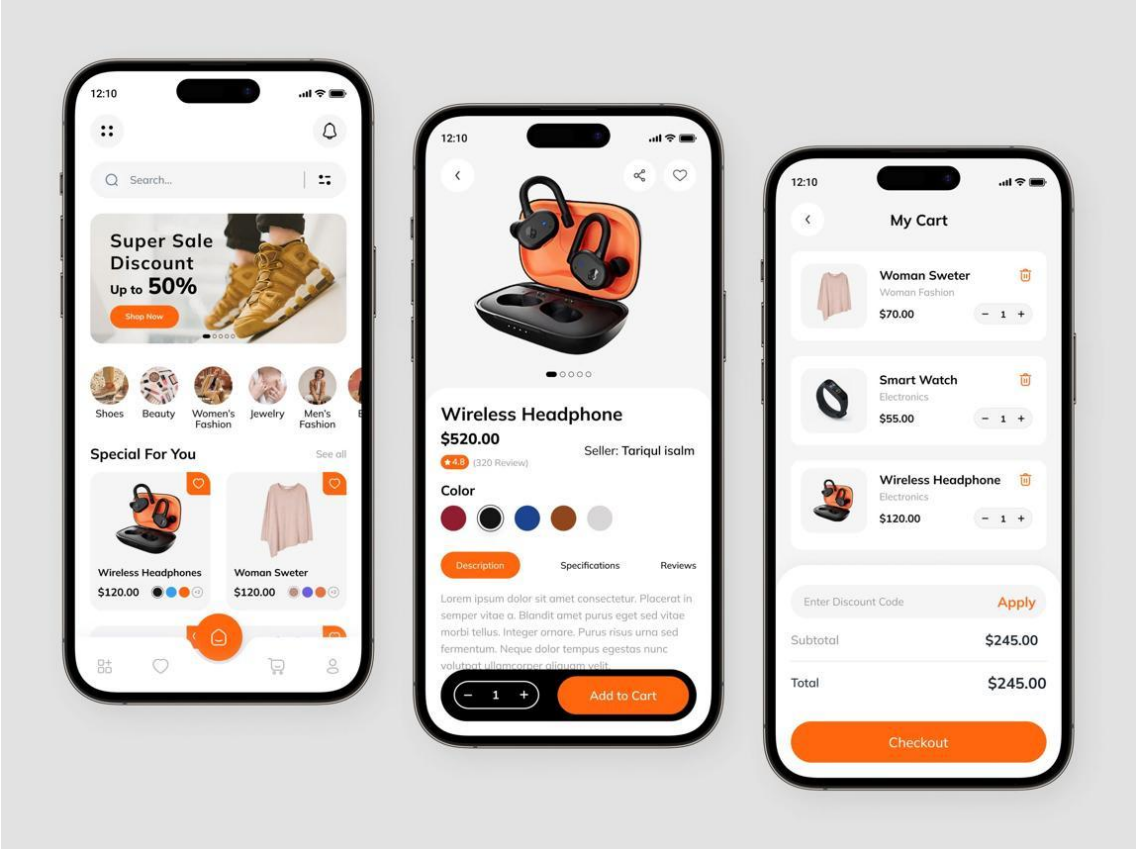
7.3. Como realizar o planejamento de releases

1. **Definir a visão e o objetivo:** identificar o problema do cliente e o resultado de negócio que a release deve proporcionar.
2. **Priorizar funcionalidades:** ordenar os itens do Product Backlog pelo valor esperado, risco, urgência e dependências.
3. **Estimar o trabalho:** utilizar estimativas relativas e informações sobre a capacidade histórica das equipes.
4. **Projetar cenários:** calcular possibilidades de entrega considerando diferentes velocidades e incertezas, em vez de depender de uma única previsão.
5. **Identificar riscos e dependências:** analisar integrações, segurança, requisitos regulatórios, migrações e necessidades operacionais.
6. **Planejar a disponibilização:** preparar implantação, comunicação, suporte, monitoramento e eventuais estratégias de reversão.
7. **Revisar continuamente:** atualizar o plano após cada Sprint, utilizando o que foi aprendido e os resultados efetivamente obtidos.

7.4. Exemplo prático de planejamento de releases

Imagine que uma empresa esteja desenvolvendo uma plataforma de comércio eletrônico. Ela deseja disponibilizar uma primeira versão com catálogo, carrinho e pagamento.

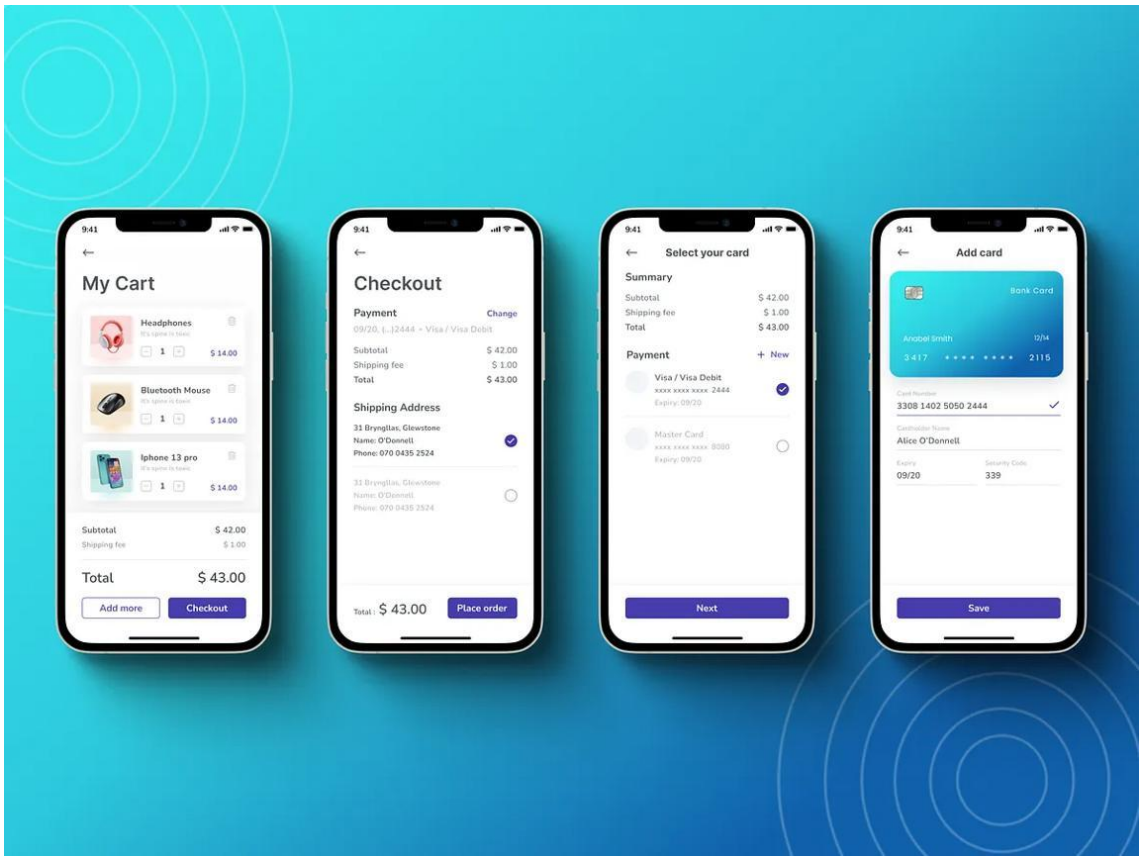
Release 1 — Descoberta e seleção de produtos



Objetivo: permitir que o cliente encontre produtos.

- Catálogo e detalhes dos produtos.
- Pesquisa e filtros.
- Testes de navegação e acessibilidade.

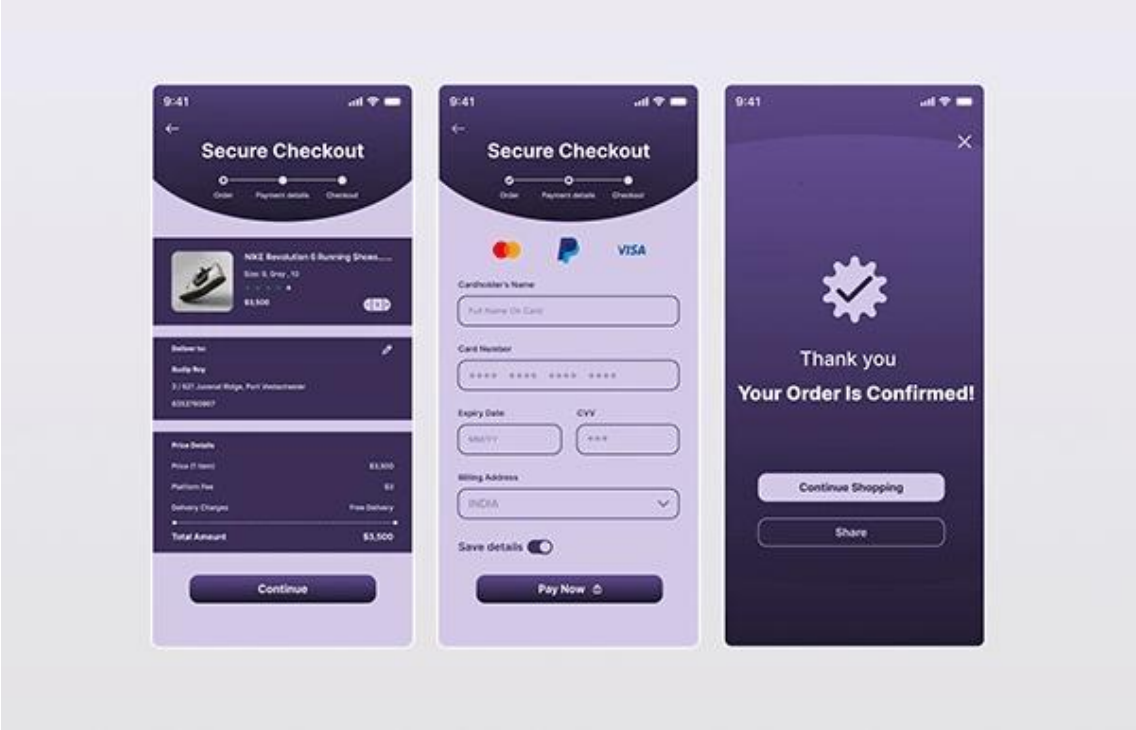
Release 2 — Preparação da compra



Objetivo: permitir que o cliente prepare seu pedido.

- Adição e remoção de produtos do carrinho.
- Atualização de quantidades.
- Cálculo do valor total e do frete.

Release 3 — Finalização do pedido



Objetivo: permitir que o cliente finalize a compra.

- Integração com o provedor de pagamento.
- Confirmação do pedido.
- Testes de segurança, falhas e recuperação.

Esse exemplo representa uma possível estratégia de evolução. Os itens e a divisão das releases devem ser adaptados à arquitetura, às necessidades dos usuários e aos riscos reais. Cada incremento produzido deve atender aos critérios de qualidade aplicáveis.

7.5. Velocidade e previsão de releases

A velocidade histórica pode ajudar a estimar quantas Sprints serão necessárias para concluir determinado conjunto de itens.

Suponha que:

- O backlog priorizado para a release tenha 110 Story Points.
- A velocidade média observada seja de 22 Story Points por Sprint.

Uma estimativa simplificada seria:

11022=5 Sprints22110=5 Sprints

Portanto, seriam aproximadamente cinco Sprints se o tamanho do backlog, a velocidade e as condições de trabalho permanecessem comparáveis.

Essa conta é apenas uma projeção pontual. Não considera variações da velocidade, mudanças de escopo, dependências ou incertezas. Para decisões importantes, é melhor trabalhar com cenários e intervalos de previsão.

7.6. Release não é necessariamente o fim de uma Sprint

Uma release pode ocorrer ao final de uma Sprint, no meio de uma Sprint ou de forma contínua, desde que os requisitos de qualidade e implantação sejam atendidos.

No Scrum, o Incremento deve estar utilizável e atender à Definition of Done. A decisão de disponibilizá-lo para usuários pode depender de estratégia comercial, riscos, controles operacionais ou outros critérios do produto.

Por isso, **concluir uma Sprint, concluir uma release e disponibilizar uma funcionalidade são acontecimentos relacionados, mas não necessariamente simultâneos**.

8. Diferenças entre Sprint Planning, Story Points, Planning Poker e Release Planning

Embora estejam relacionados, esses conceitos possuem finalidades diferentes.

Conceito	Principal finalidade	Horizonte
Sprint Planning	Definir o objetivo, o trabalho selecionado e o plano da Sprint.	Uma Sprint
Story Points	Estimar o tamanho relativo dos itens.	Item do backlog
Planning Poker	Facilitar a estimativa colaborativa dos itens.	Durante o refinamento ou planejamento
Planejamento de Sprint	Organizar a execução do trabalho durante a iteração.	Curto prazo
Release Planning	Planejar a evolução e a disponibilização de funcionalidades.	Várias Sprints ou um horizonte de produto

A relação entre eles pode ser resumida assim:

Objetivos do produto e prioridades

O que gera valor para o cliente?

Refinamento e estimativas

Story Points e Planning Poker

Sprint Planning

Sprint Goal, seleção de itens e plano de execução

Desenvolvimento e inspeção

Incrementos, feedback e melhoria contínua

Planejamento e evolução das releases

Disponibilização de valor e revisão das previsões

O fluxo não é totalmente linear. As informações obtidas durante uma Sprint podem alterar as prioridades do produto, as estimativas e o planejamento das releases futuras.

9. Exemplo integrado: aplicação dos conceitos em um único projeto

Imagine uma equipe responsável por criar um aplicativo de agendamento de consultas.

O objetivo do produto é permitir que pacientes encontrem profissionais, selecionem horários e realizem agendamentos.

Etapa 1 — Organizar o Product Backlog

O Product Owner prioriza as seguintes histórias:

História	Descrição	Story Points
US01	Consultar profissionais disponíveis	3
US02	Visualizar horários livres	3
US03	Agendar uma consulta	5
US04	Cancelar um agendamento	3
US05	Receber confirmação por e-mail	5

Os valores são ilustrativos e devem ser definidos pela equipe.

Etapa 2 — Aplicar Planning Poker

A equipe utiliza Planning Poker para estimar cada história.

Durante a discussão de US03, alguns participantes consideram o agendamento simples, enquanto outros identificam riscos relacionados à concorrência entre usuários que tentam reservar o mesmo horário.

A discussão revela a necessidade de esclarecer as regras de disponibilidade e evitar agendamentos duplicados. Depois desse esclarecimento, a equipe define a estimativa de 5 pontos.

Etapa 3 — Realizar o Sprint Planning

A equipe define o Sprint Goal:

Permitir que o paciente consulte profissionais e visualize horários disponíveis para atendimento.

Seleciona US01 e US02, totalizando 6 pontos.

Os Developers elaboram um plano inicial que inclui:

- Implementação da consulta de profissionais.
- Integração com os dados de disponibilidade.
- Testes de filtros e horários.
- Validação dos critérios de aceitação.

A seleção é coerente com o objetivo, desde que o trabalho possa ser concluído dentro da Sprint.

Etapa 4 — Executar a Sprint

Durante a Sprint, os Developers acompanham o progresso e atualizam o Sprint Backlog.

Ao descobrir que a integração com a agenda exige ajustes, a equipe adapta seu plano de execução. O objetivo continua sendo entregar uma maneira confiável de consultar profissionais e horários.

Ao final, a equipe inspeciona o incremento e verifica se os critérios de conclusão foram atendidos.

Etapa 5 — Planejar a release

Após a Sprint, o Product Owner considera os resultados e prioriza a capacidade de agendar consultas.

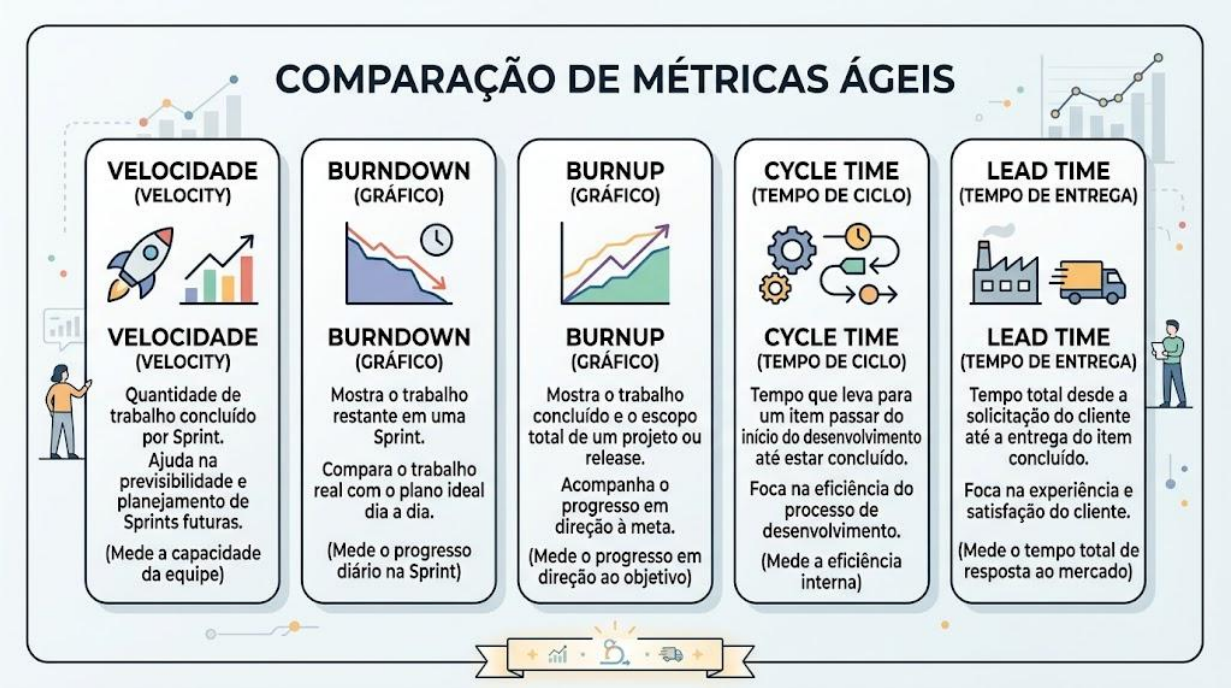
Uma possível sequência de entregas seria:

- Primeira entrega: consulta de profissionais e horários.
- Segunda entrega: agendamento e confirmação.
- Terceira entrega: cancelamento e melhorias de comunicação.

O plano é revisto à medida que as Sprints avançam. Caso surjam novas necessidades dos pacientes ou riscos técnicos, as prioridades podem mudar.

Esse exemplo mostra como estimativas, planejamento de Sprints e planejamento de releases se complementam para reduzir incertezas e entregar valor progressivamente.

10- Métricas Ágeis



- **Velocity:** Mede a quantidade de pontos concluídos por Sprint, garantindo previsibilidade.
- **Burndown Chart:** Mostra o trabalho restante até o final da Sprint.
- **Burnup Chart:** Exibe o trabalho concluído em relação ao escopo total do projeto.
- **Cycle Time:** Mede a eficiência interna (do início do desenvolvimento até a conclusão).
- **Lead Time:** Mede o tempo total da ponta a ponta (desde o pedido do cliente até a entrega final).

1. Introdução às Métricas Ágeis

O Fim das Métricas de Vaidade: No gerenciamento tradicional de projetos, o progresso costuma ser medido por métricas superficiais ou de volume de esforço, como quantidade de linhas de código escritas, horas gastas na mesa ou páginas de documentação geradas. No contexto ágil, a lógica muda drasticamente: o foco absoluto é medir **valor entregue**, **previsibilidade** e **eficiência do fluxo de trabalho**.

O Propósito das Métricas: As métricas ágeis não servem para punir a equipe ou microgerenciar a produtividade individual de cada funcionário. Elas funcionam como o "painel de bordo" de um carro de corrida: ajudam o time a entender quão rápido estão indo, se estão gastando combustível no ritmo certo e se vão

conseguir chegar ao destino no prazo previsto, permitindo correções de rota antecipadas.

2. Velocity (Velocidade)

O que é: É uma métrica clássica que mede a quantidade de trabalho que a equipe consegue concluir dentro de um ciclo de desenvolvimento (geralmente uma *sprint*).
Como é medida: É calculada somando os **pontos de história** (unidades relativas de tamanho/complexidade das tarefas) de todos os itens do backlog que foram 100% concluídos e entregues ao final da *sprint*. Por exemplo, se a equipe concluiu três itens pontuados em 3, 5 e 5, o *Velocity* daquela *sprint* foi de 13 pontos.
Para que serve: Ajuda fundamentalmente na **previsibilidade**. Com o histórico de *velocities* das *sprints* anteriores (ex: média das últimas 5 *sprints*), o time consegue prever com muito mais precisão quanto trabalho conseguirá absorver nas próximas entregas, facilitando o planejamento de lançamentos futuros.
⚠️ **Alerta importante para provas:** O *Velocity* **não** serve para comparar equipes diferentes (já que a pontuação é subjetiva de cada time), nem deve ser usado como meta de pressão corporativa para forçar a equipe a trabalhar mais rápido de forma insustentável.

3. Burndown Chart (Gráfico de Queima)

O que é: É um gráfico visual que mostra a **quantidade de trabalho restante** em relação ao tempo previsto (geralmente exibido para uma *sprint* ou para um projeto inteiro).
Como funciona o eixo do gráfico:
O eixo vertical (*Y*) representa o trabalho restante (medido em horas, tarefas ou pontos de história).
O eixo horizontal (*X*) representa os dias decorridos do projeto ou *sprint*.
A Linha Ideal vs. A Linha Real: No gráfico existe uma linha diagonal teórica (a "meta ideal") que desce uniformemente até zerar no último dia. A linha real mostra o andamento diário da equipe.
Para que serve: Permite que qualquer pessoa da equipe bata o olho e saiba instantaneamente se o projeto está **no prazo, adiantado ou atrasado**. Se a linha real estiver muito acima da linha ideal, é um sinal de alerta de que o escopo pode não ser entregue a tempo, permitindo que o time aja imediatamente.

4. Burnup Chart (Gráfico de Ascensão)

O que é: Ao contrário do *Burndown* (que foca no que falta terminar), o *Burnup Chart* foca no **trabalho já concluído**, exibindo também se houve mudanças no escopo total ao longo do caminho.

Como funciona: O gráfico possui duas linhas principais que sobem no eixo vertical ao longo do tempo:
Uma linha na parte inferior que mostra o **trabalho acumulado e concluído** (quanto já foi entregue).
Uma linha na parte superior que mostra o **escopo total** do projeto.
Para que serve: É excelente para dar transparência à **evolução real da entrega** e deixar muito clara a "variabilidade do escopo". Se a linha superior (escopo total) começar a subir constantemente, significa que novos requisitos estão sendo adicionados ao projeto, mostrando visualmente o impacto dessas mudanças na data final de entrega.

5. Lead Time e Cycle Time

Estes dois conceitos medem a **eficiência temporal** do fluxo de valor, sendo fundamentais para entender a velocidade de entrega:

Lead Time (Tempo de Espera / De ponta a ponta):
O que é: É o tempo total que leva desde o exato momento em que uma ideia ou demanda é solicitada pelo cliente/negócio até o momento em que ela é totalmente desenvolvida, testada e entregue em produção nas mãos do usuário final.
O que mede: A agilidade percebida pelo cliente externo (quanto tempo ele espera entre pedir algo e ver aquilo funcionando).
Cycle Time (Tempo de Ciclo):
O que é: É o tempo que leva para uma tarefa ser concluída **desde o momento em que a equipe realmente começa a trabalhar nela** (mão na massa) até a sua conclusão final.
O que mede: A eficiência interna do processo produtivo da equipe.
Resumo prático para não confundir: O *Lead Time* começa no pedido do cliente (incluindo o tempo que a tarefa ficou parada na fila de espera). O *Cycle Time* só começa quando o desenvolvedor puxa a tarefa para o "fazendo" (*in progress*). O *Lead Time* é sempre maior ou igual ao *Cycle Time*.

FONTES UTILIZADAS

- https://www.scrum.org/resources/evidence-based-management
- https://www.atlassian.com/agile/project-management/metrics
- https://kanban.university/

11- Qualidade em Projetos Ágeis



A imagem é composta por **três blocos visuais simples**:

1. **Qualidade:**
    a. **Software Robusto e Confiável:** Foco em entregas seguras.
    b. **Código Limpo:** Facilidade de manutenção.
    c. **Menos Defeitos (Bugs):** Redução de erros no sistema.
    d. **Foco no Usuário:** Boa experiência para o cliente final.
2. **DoD (Definition of Done):**
    a. **Critérios Concluídos:** Atendimento aos requisitos.
    b. **Testes Aprovados:** Testes de unidade e integração bem-sucedidos.
    c. **Revisão de Código:** Código verificado pelo time.
    d. **Publicação em Teste:** Sistema pronto em ambiente de homologação.
3. **CI / Refatoração:**
    a. **Integração Contínua (CI):** Automação de compilação e testes.
    b. **Refatoração Frequente:** Melhoria contínua da estrutura do código sem alterar seu funcionamento externo.

1. Introdução à Qualidade na Agilidade

- **O Paradigma da Qualidade:** No modelo tradicional de desenvolvimento, a qualidade era tratada como uma etapa isolada e burocrática que acontecia apenas no final do projeto (uma fase inteira dedicada exclusivamente a testes). Na agilidade, a qualidade **não é uma fase final, mas sim uma prática contínua** embutida em todas as etapas do processo de desenvolvimento.
- **O Foco na Prevenção:** Em vez de apenas caçar bugs depois que o código está pronto, a agilidade utiliza técnicas de engenharia e alinhamento de critérios para garantir que o software nasça correto, limpo e testável desde o primeiro dia.

2. Definition of Done (DoD - Definição de Pronto)

- **O que é:** É um acordo explícito e compartilhado por toda a equipe sobre o que significa uma tarefa estar **realmente finalizada**. Enquanto o *Definition of Ready* (DoR) define quando uma tarefa está pronta para *começar*, a DoD define quando ela pode ser considerada 100% concluída.
- **Exemplos de critérios comuns na DoD:**
    - O código foi escrito e revisado por outro colega (*Code Review*).
    - Todos os testes automatizados (unitários e de integração) passaram com sucesso.
    - O código foi integrado ao repositório principal (*CI*).
    - A documentação técnica foi atualizada.
    - O item foi testado e aprovado em ambiente de homologação.
- **Para que serve:** Evita a famosa ilusão do desenvolvedor que diz *"meu código está pronto, só falta testar"*. Na agilidade, se não passou por todos os critérios da DoD, a funcionalidade **não está pronta** e não pode ser entregue.

3. Testes Automatizados

- **O que são:** São scripts de software criados especificamente para testar partes do sistema de forma automática, rodando em frações de segundo por meio de comandos ou ferramentas de integração.
- **Por que são vitais na agilidade:** Como os ciclos de entrega (*sprints*) são extremamente curtos, seria humanamente impossível testar todo o sistema manualmente a cada nova alteração sem perder prazos e deixar escapar bugs críticos.
- **Principais tipos básicos:**
    - **Testes Unitários:** Testam a menor parte isolada do código (como uma função ou método matemático específico).

    - **Testes de Integração:** Verificam se diferentes módulos ou componentes do sistema conversam e funcionam bem juntos.
    - **Testes de Aceitação / Sistema:** Validam se o comportamento geral do sistema atende às regras de negócio esperadas pelo usuário.

4. Integração Contínua (CI - Continuous Integration)

- **O que é:** É uma prática de desenvolvimento de software em que os membros da equipe integram e juntam seus códigos em um repositório central compartilhado várias vezes ao dia (ou pelo menos diariamente).
- **Como funciona na prática:** Cada vez que um desenvolvedor envia (*push*) seu código novo para o repositório central, um servidor automatizado dispara uma série de rotinas para compilar o sistema e rodar todos os testes automatizados instantaneamente.
- **Para que serve:** Evita o pesadelo clássico do "inferno da integração", que ocorria no modelo tradicional quando os programadores passavam meses codificando isoladamente e tentavam juntar tudo no último dia, gerando conflitos gigantescos e quebrando o sistema inteiro. A CI avisa a equipe imediatamente se o código novo quebrou alguma parte do sistema existente.

5. Refatoração

- **O que é:** É o processo de **reestruturar e limpar um código fonte existente** sem alterar o seu comportamento externo ou as funcionalidades que ele entrega ao usuário.
- **O objetivo principal:** Melhorar a legibilidade do código, reduzir a complexidade lógica, eliminar duplicações e facilitar futuras manutenções.
- **Analogia simples:** É como arrumar, limpar e organizar uma gaveta de ferramentas bagunçada para que você consiga encontrar e usar os utensílios muito mais rápido na próxima vez, sem mexer no que as ferramentas fazem.
- **Relação com a agilidade:** O código em projetos ágeis evolui de forma orgânica. Refatorar constantemente garante que o sistema continue flexível e saudável ao longo do tempo.

6. Dívida Técnica (*Technical Debt*)

- **O que é:** É um conceito cunhado por Ward Cunningham que descreve o custo implícito de escolher uma solução de desenvolvimento rápida, "gambiarra" ou de baixa qualidade a curto prazo, em vez de aplicar a solução arquitetural correta e limpa.

- **Por que ela surge:** Muitas vezes, para entregar uma funcionalidade correndo e aproveitar uma oportunidade de mercado (*time-to-market*), a equipe toma um atalho no código.
- **O Perigo do Acúmulo:** Assim como uma dívida financeira com juros, se a equipe acumula muita dívida técnica sem reservação de tempo para limpá-la (refatoração), o código vai se tornando podre, frágil e lento de modificar. Com o tempo, a equipe passa a gastar mais tempo consertando problemas antigos do que criando novos valores para o cliente.

FONTES UTILIZADAS

- https://martinfowler.com/articles/continuousIntegration.html
- https://martinfowler.com/bliki/TechnicalDebt.html
- https://refactoring.guru/refactoring
- https://www.scrum.org/resources/blog/done-understanding-definition-done
- https://engsoftmoderna.info/cap9.html